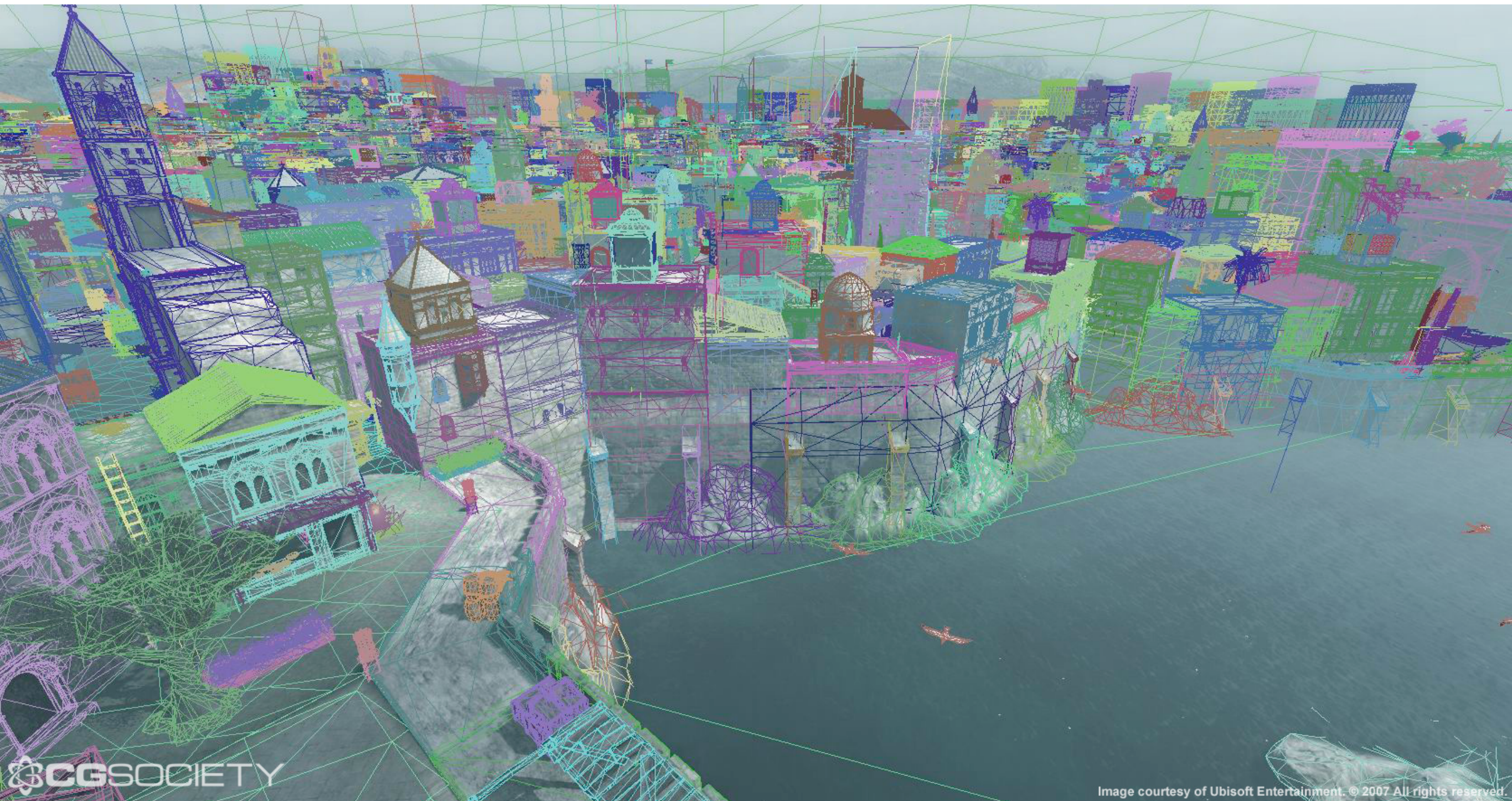


Representation of shape in graphics

Representation of shape



Representation of shape



Representation of shape



3D object representations

Raw data	Surfaces	Solids	High-level structures
Point cloud	Mesh	Voxel	Scene graph
	Subdivision	CSG	Skeleton
	Parametric	Sweep	
	Implicit		

3D object representations

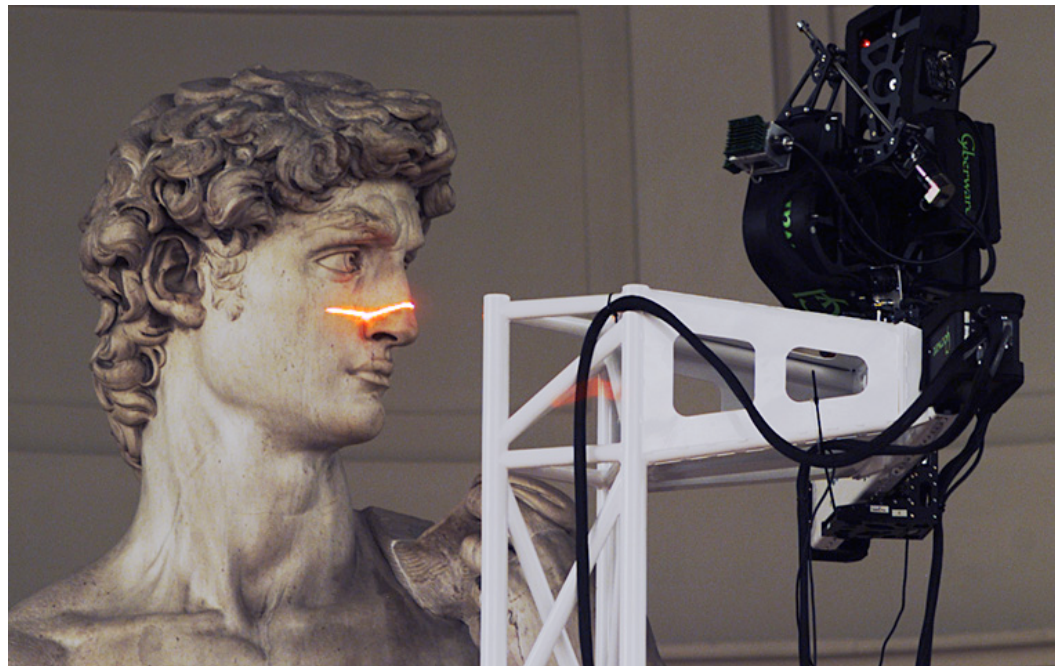
- **Triangle meshes** - the most used representation
- **Subdivision surfaces** - movies
- **Spline patches** - modeling
- **Constructive Solid Geometry (CSG)** - modeling machine parts
- **Volume data** - medical imaging
- **Point clouds** - computer vision

3D object representations

Raw data	Surfaces	Solids	High-level structures
Point cloud	Mesh	Voxel	Scene graph
	Subdivision	CSG	Skeleton
	Parametric	Sweep	
	Implicit		

Point cloud

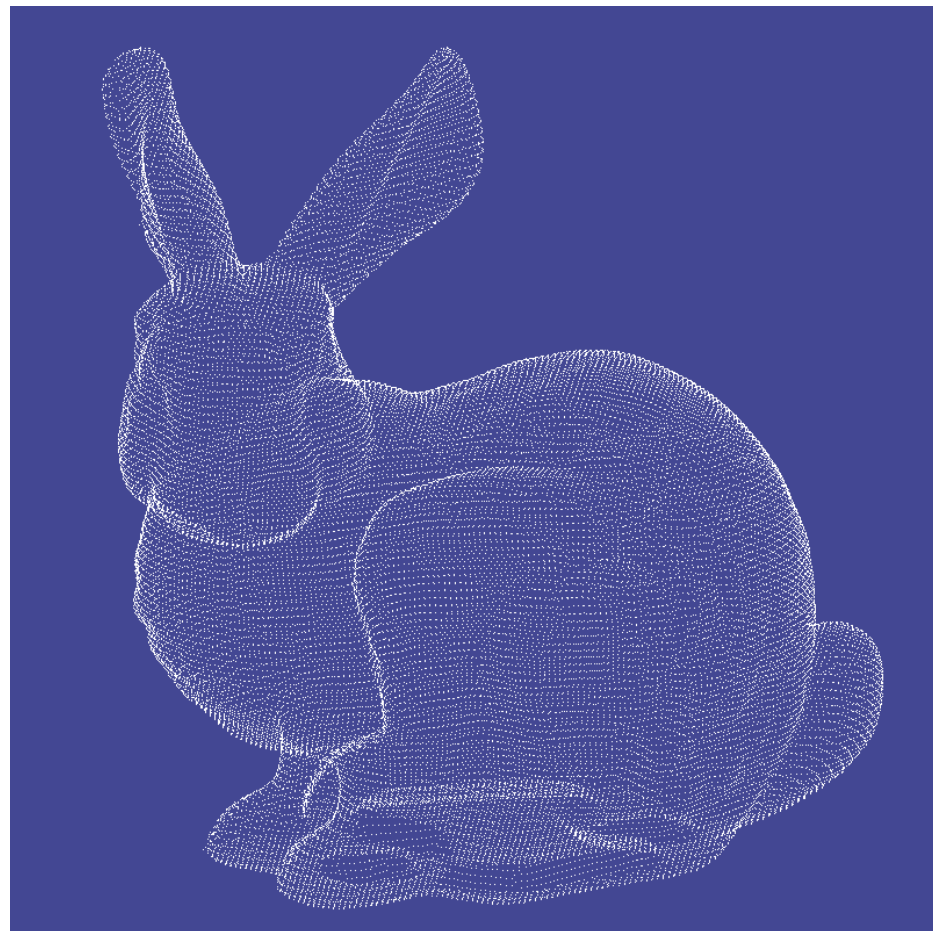
- Simplest representation, store only points without connectivity between them
- Collection of 3D coordinates (in some cases also the normals)
- Created using **3d scanning devices**



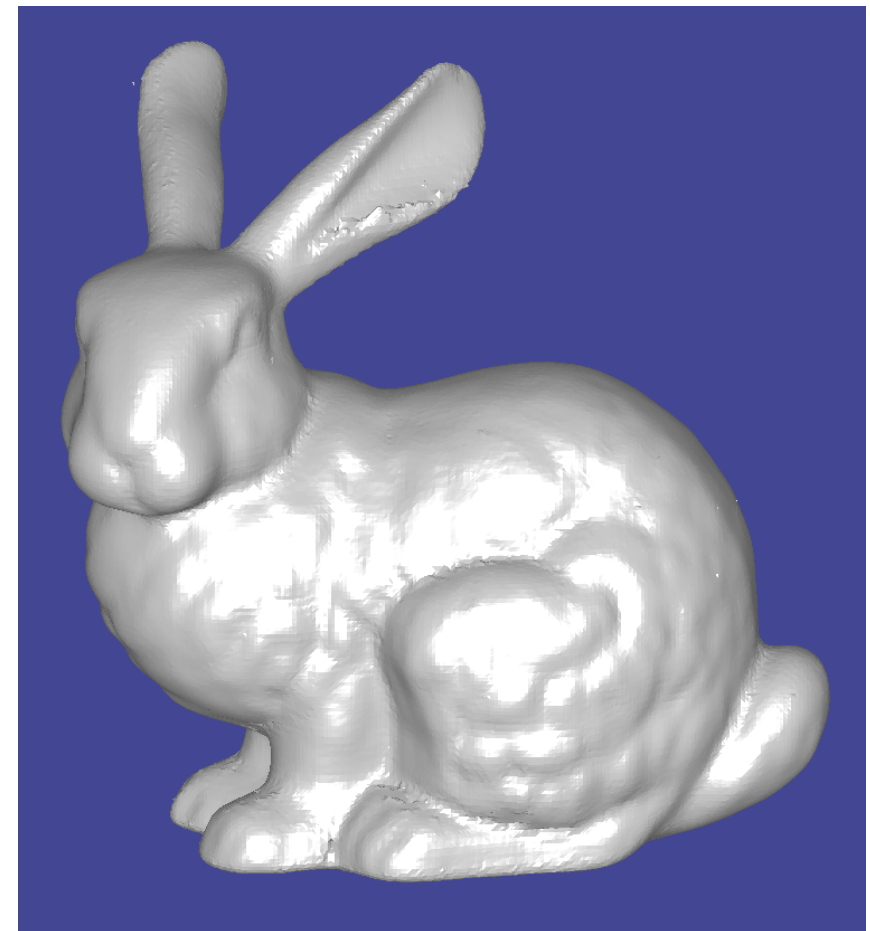
[Stanford, The Digital Michelangelo Project]

3D Point Cloud Reconstruction

- Issues: unstructured data
- Construct a polygonal (e.g. triangle mesh) representation of the point cloud
- 362,272 points (about 725,000 triangles) -> 35,947 vertices, 69,451 triangles



[http://www.ccs.neu.edu/home/gaob/images/bunny_pc.png]

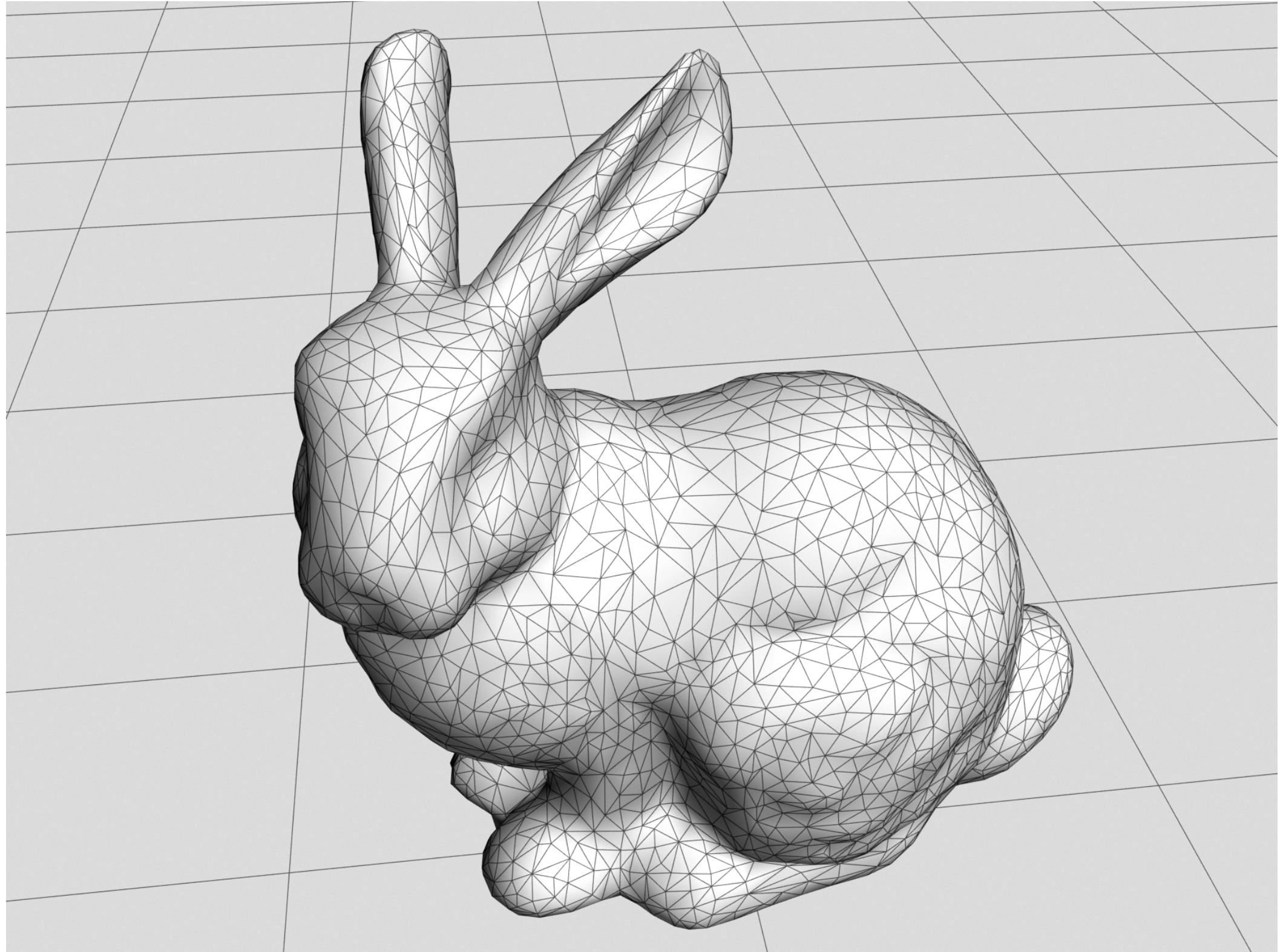


[http://www.ccs.neu.edu/home/gaob/images/bunny_mc_s.png]

3D object representations

Raw data	Surfaces	Solids	High-level structures
Point cloud	Mesh	Voxel	Scene graph
	Subdivision	CSG	Skeleton
	Parametric	Sweep	
	Implicit		

Polygonal mesh



Boundary representation

- Defined by:
 - **Geometric description**: specifies vertex coordinates, surface equations, etc.
 - **Topological description**: specifies connectivity of vertices, face orientation, etc.
- **Polygonal mesh**
 - Represent object surfaces using polygons
 - Uses different primitives: **vertices**, **edges**, and **faces**
 - Usually the 3D objects are represented by **triangle meshes**

Triangle mesh

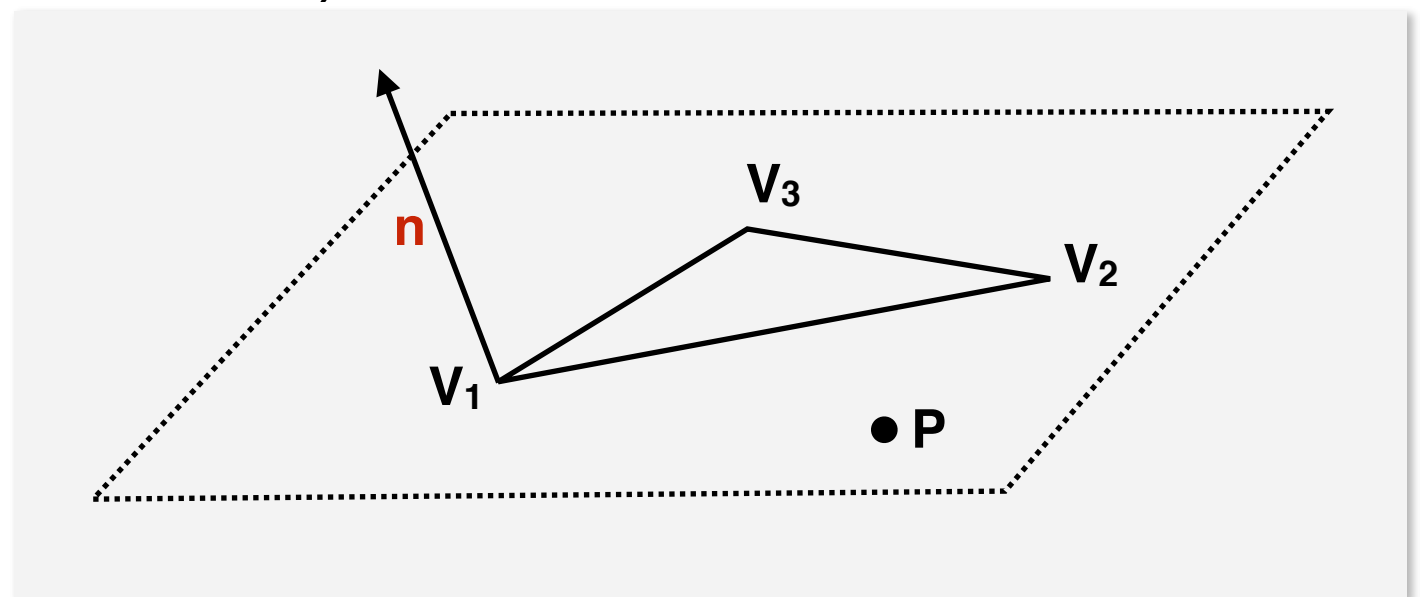
- Operations applied to triangle meshes:
 - **Subdivision**
 - replace a triangle by several smaller triangles
 - used to smooth out a mesh that has sharp points or edges
 - repeated subdivision operations increase the triangle count substantially; this can have a major impact on rendering performance
 - **Simplification**
 - a mesh is replaced by another mesh that's similar to it, topologically or geometrically

Triangles

- Always convex
- Fixed number of vertices
- The three vertices that define the triangle are on the same plane (no need to check planarity)
- Easy method to compute the normal vector (used to identify front/back faces)

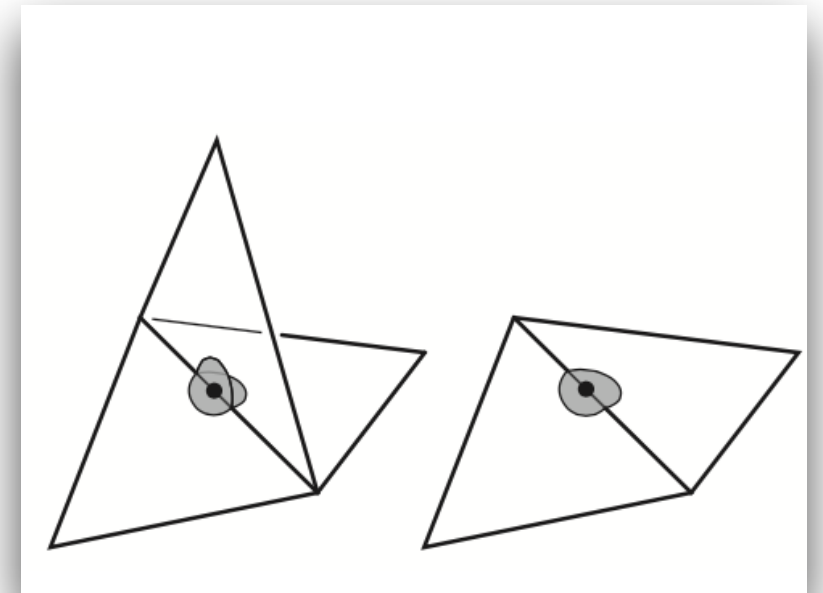
$$\mathbf{n} = (V_2 - V_1) \times (V_3 - V_1)$$

$$\mathbf{n} \cdot (P - V_1) = 0$$



Mesh topology

- The simplest and most restrictive requirement on the topology of a mesh is for the surface to be a manifold
- **Manifold mesh**
 - separates the space on the inside of the surface from the space outside
- Manifold **conditions**
 - Every edge is shared by exactly two triangles
 - Every vertex has a single, complete loop of triangles around it
- Manifold with **boundary conditions**
 - Every edge is used by either one or two triangles
 - Every vertex should have one connected fan of triangles

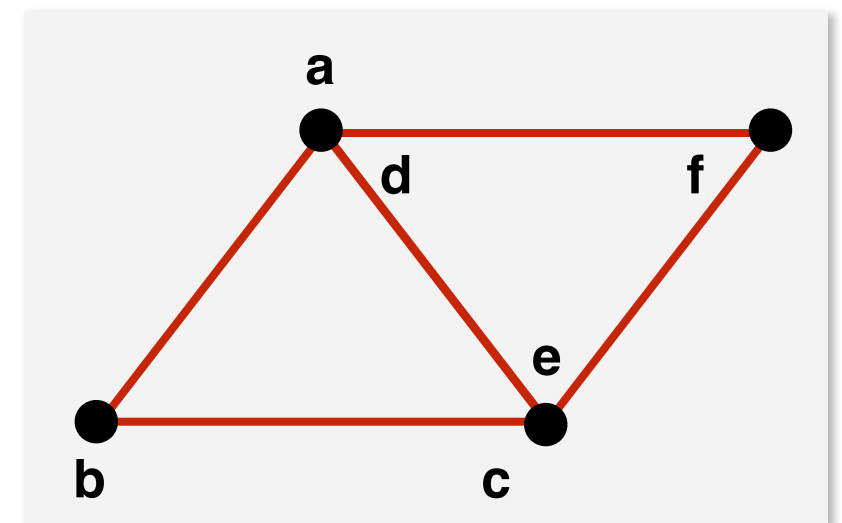


Data structures

Polygon-soup

- consist of a **list of polygons** (triangles), and each polygon contains the vertices coordinates
- very simple to render (process each individual polygon)
- **issues:**
 - multiple specifications of a particular vertex
 - no info about which triangle uses a particular vertex
 - no info about adjacent triangles

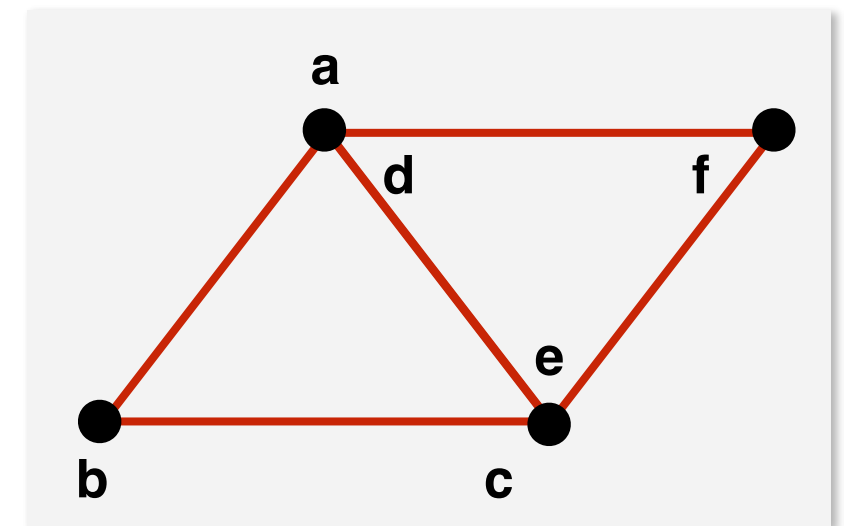
triangle #	vertex 1	vertex 2	vertex 3
0	(a _x , a _y , a _z)	(b _x , b _y , b _z)	(c _x , c _y , c _z)
1	(d _x , d _y , d _z)	(e _x , e _y , e _z)	(f _x , f _y , f _z)



Data structures

Indexed face set

- uses a list of vertices and a list of polygons
- each vertex is defined only once
- **issues:**
 - find all triangles that share a common vertex
 - find all triangles for all vertices
- store more connectivity information
 - triangles sharing a given vertex
 - triangles which are adjacent (sharing an edge)
 - vertices adjacent to a given vertex

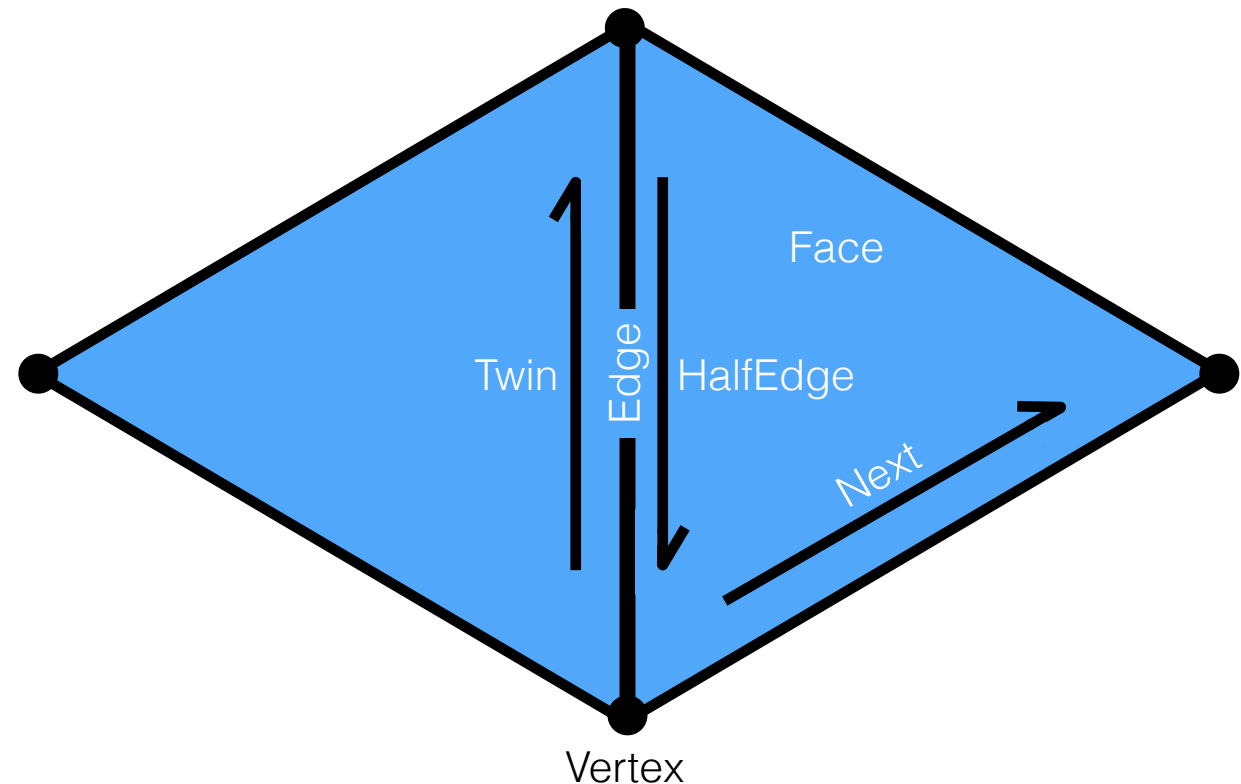


vertex #	x	y	z
0	a_x	a_y	a_z
1	b_x	b_y	b_z
2	c_x	c_y	c_z
3	f_x	f_y	f_z

triangle #	v_1	v_2	v_3
0	0	1	2
1	0	2	3

Half-edge data structure

```
struct HalfEdge
{
    HalfEdge* twin;
    HalfEdge* next;
    Vertex* vertex;
    Edge* edge;
    Face* face;
};
```



```
struct Edge
{
    HalfEdge*
    halfedge;
    ...
};
```

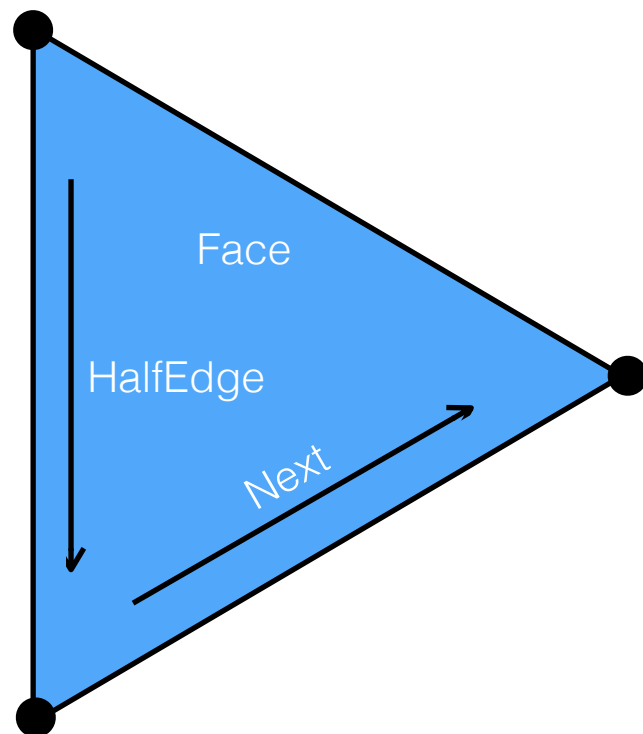
```
struct Face
{
    HalfEdge* halfedge;
    ...
};
```

```
struct Vertex
{
    HalfEdge* halfedge;
    ...
};
```


Mesh traversal

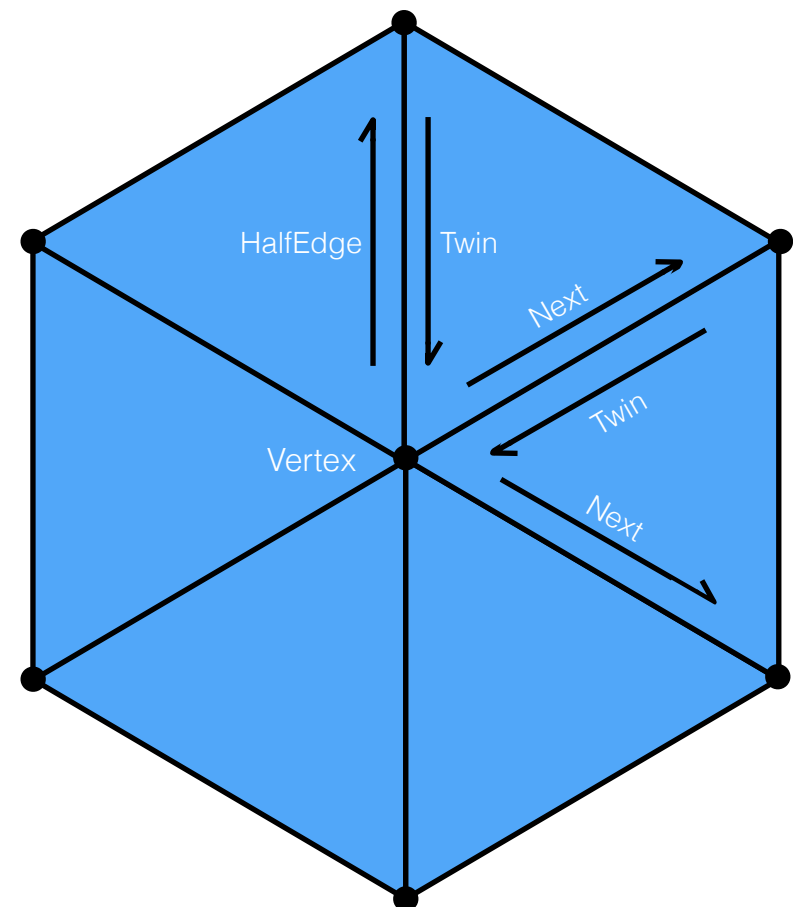
Visit all vertices of a face

```
HalfEdge* h = f->halfedge;  
do{  
    h = h->next;  
    //process h->vertex  
}while( h != f->halfedge );
```

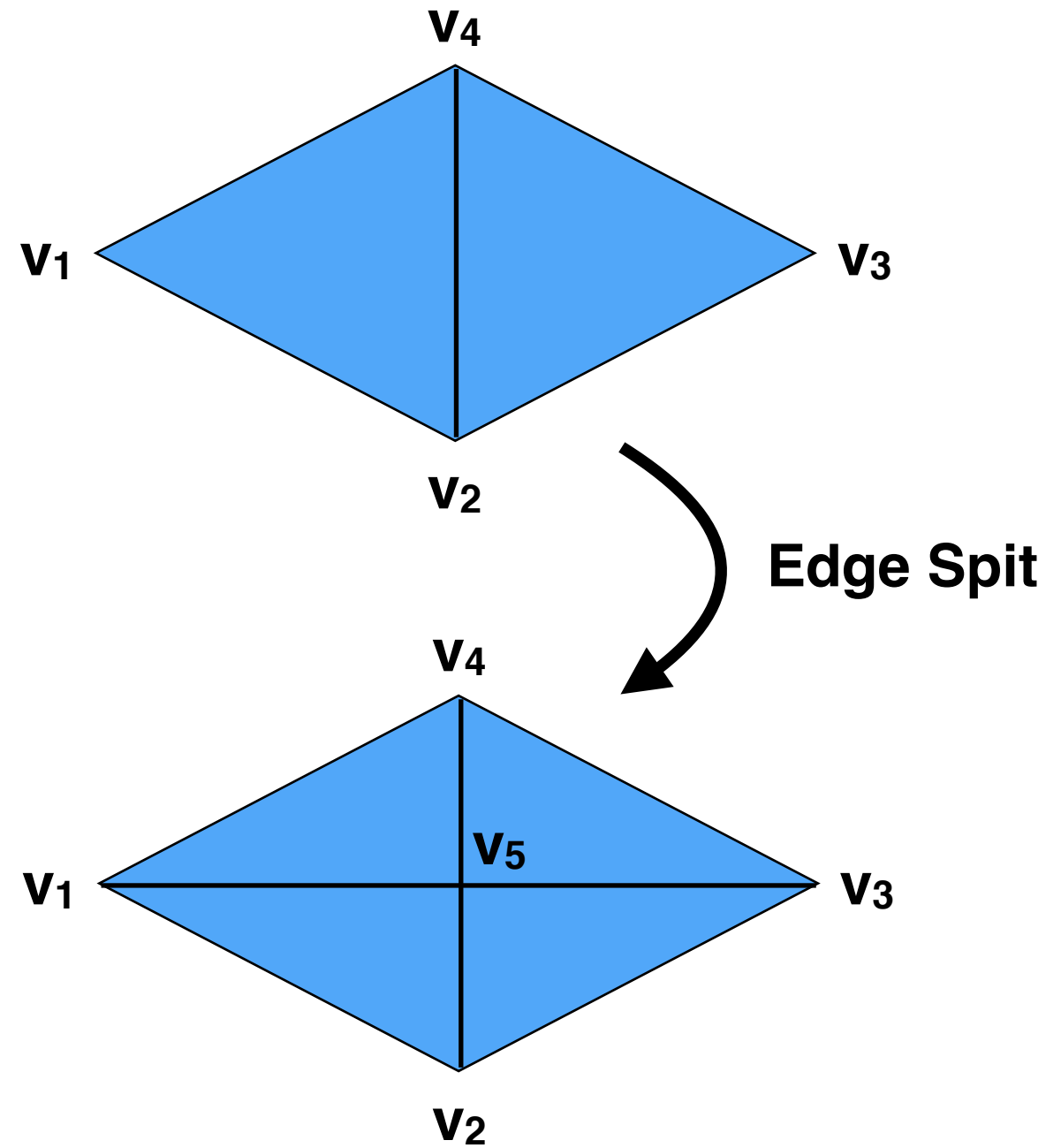
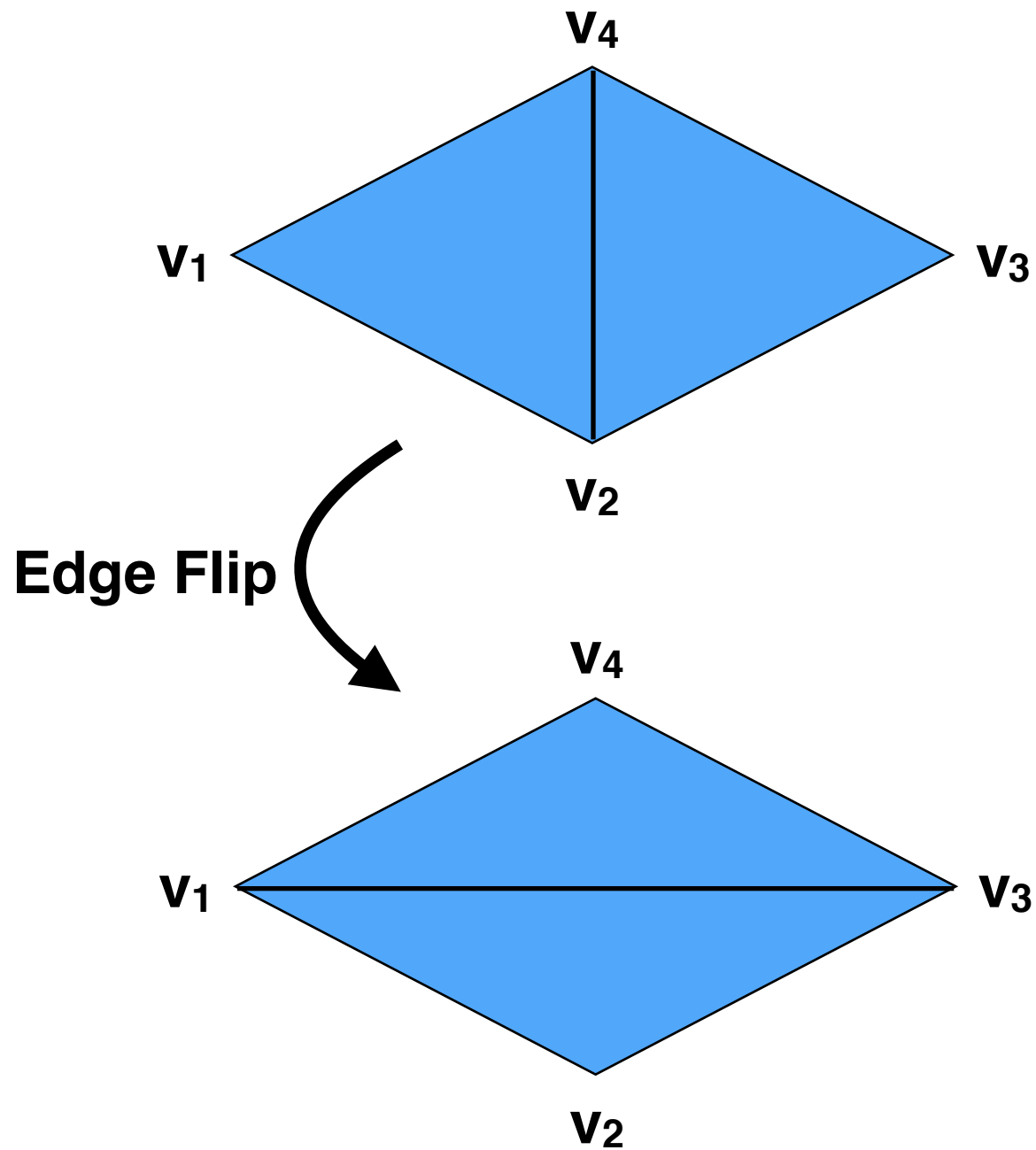


Visit all neighbors of a vertex

```
HalfEdge* h = v->halfedge;  
do{  
    h = h->twin->next;  
    //process h->vertex  
}while( h != v->halfedge );
```



Operations on HalfEdges



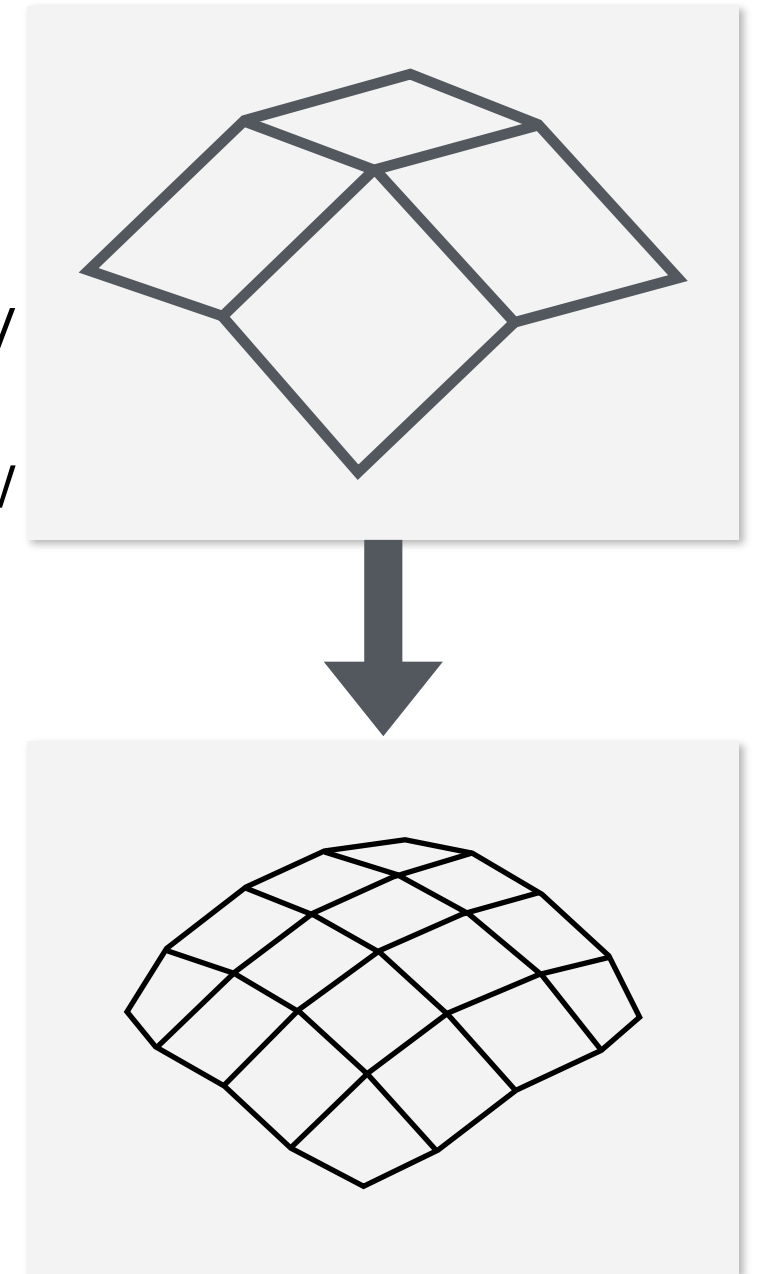
3D object representations

Raw data	Surfaces	Solids	High-level structures
Point cloud	Mesh	Voxel	Scene graph
	Subdivision	CSG	Skeleton
	Parametric	Sweep	
	Implicit		

Subdivision surfaces

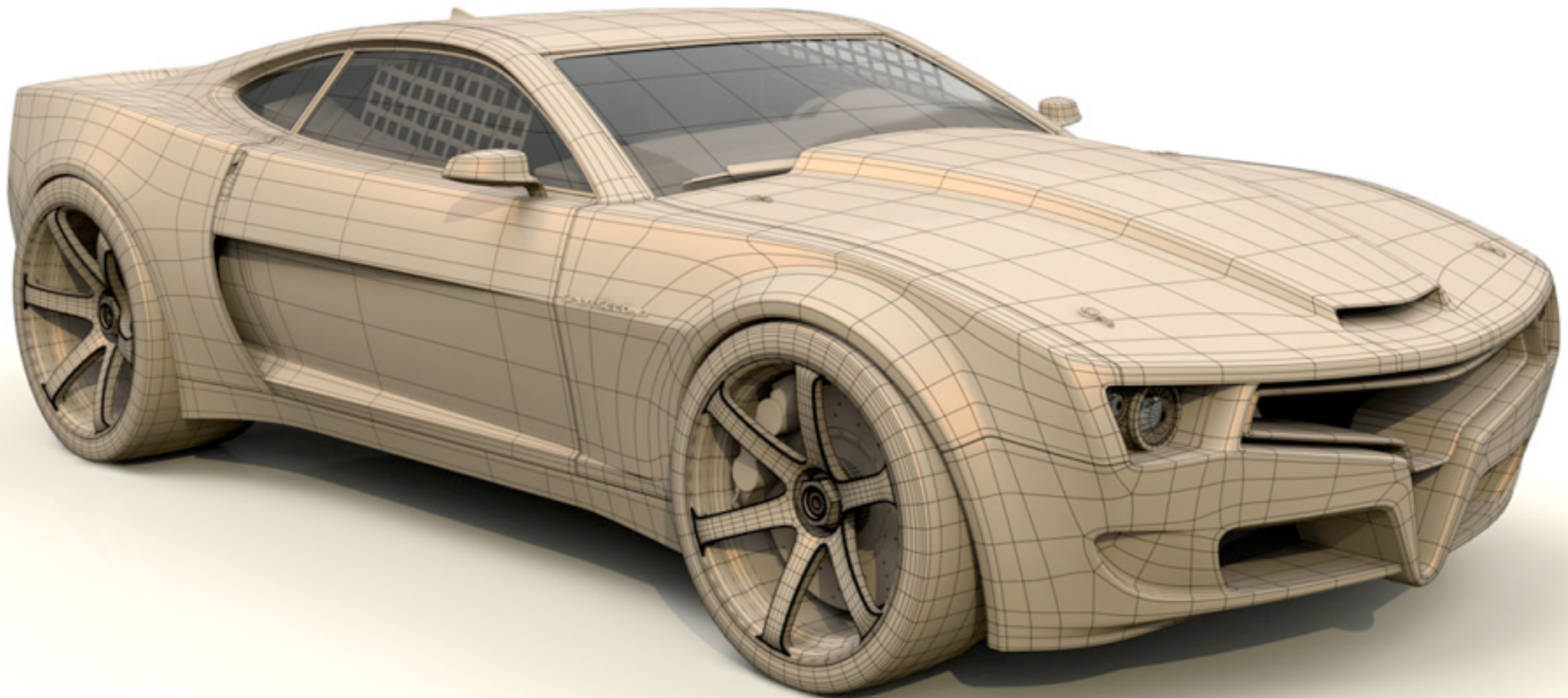
- A method for representing smooth surfaces by specifying a coarse polygonal mesh
- Subdivision surfaces are defined recursively
- Each step of refinement (iteration) adds new faces and vertices
- The surfaces converges to a smooth limit surface (the “mathematical” result after infinite refinement steps)
- Subdivision schemes

	Triangles	Quads
Approximating	Loop	Catmull-Clark
Interpolating	Butterfly	Kobbelt

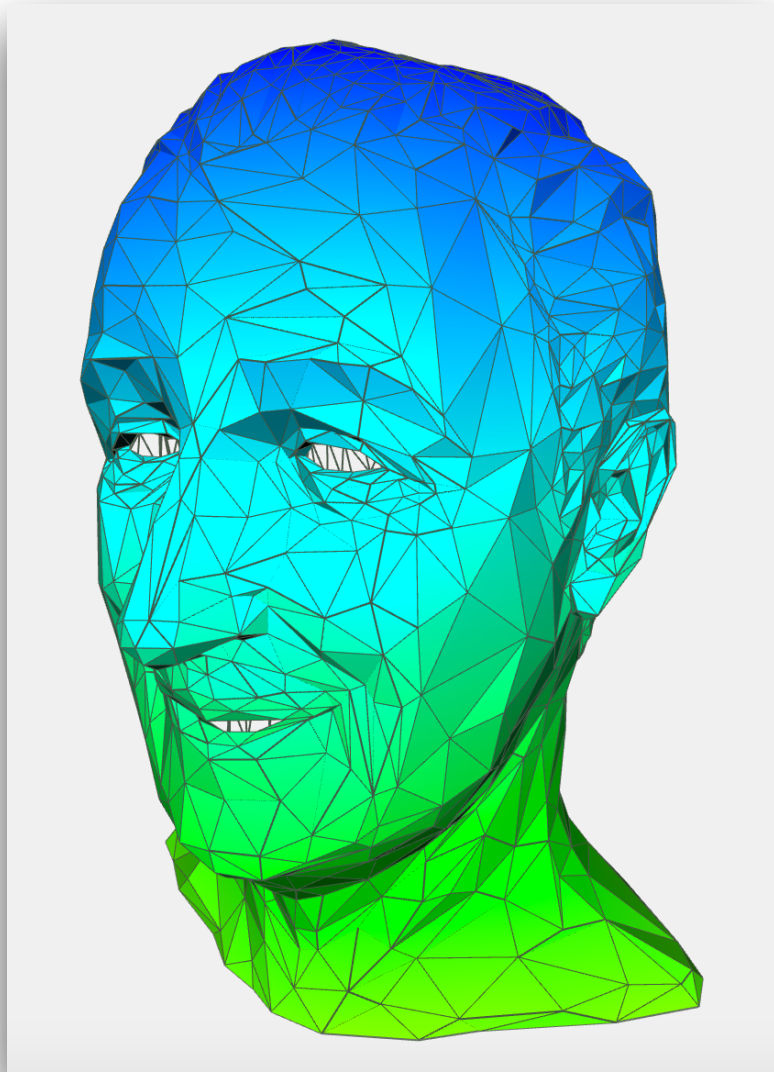


Subdivision surfaces

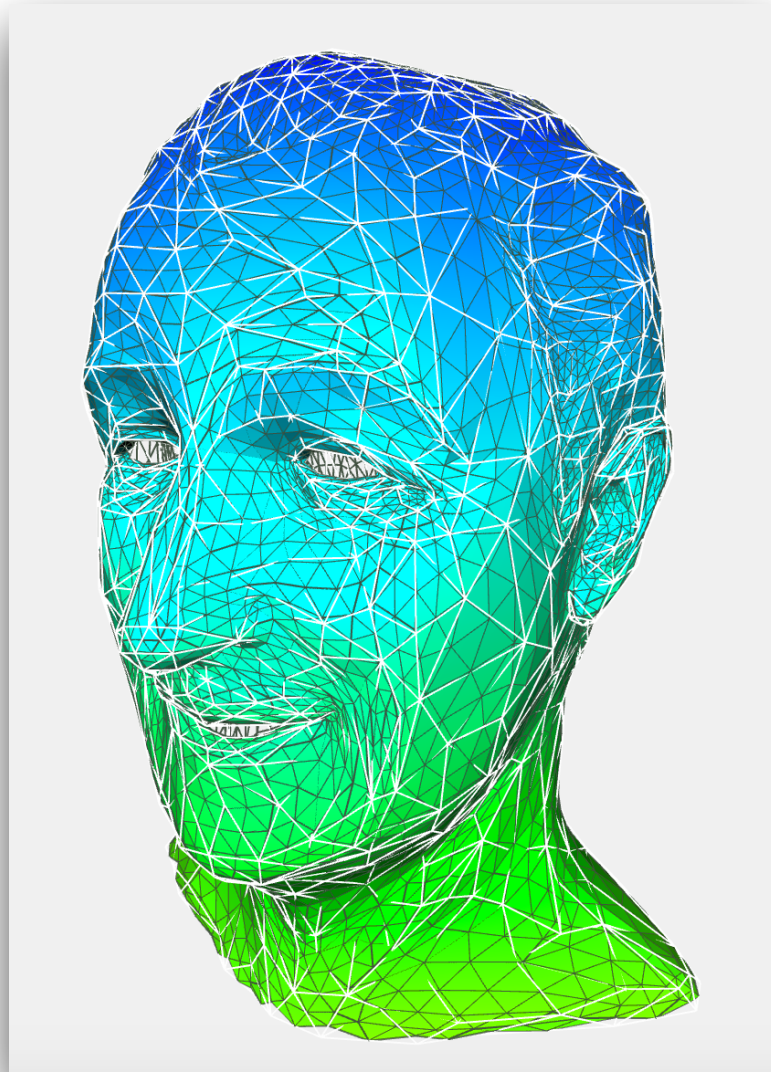
Sub-Divisional Concept Modeling



Subdivision surfaces



Vertices count: **1642**
Face count: **3232**



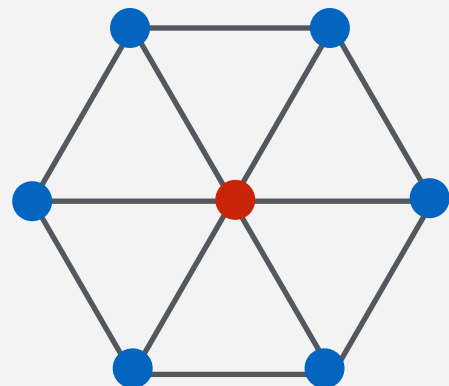
Vertices count: **6524**
Face count: **12928**



Vertices count: **25984**
Face count: **51712**

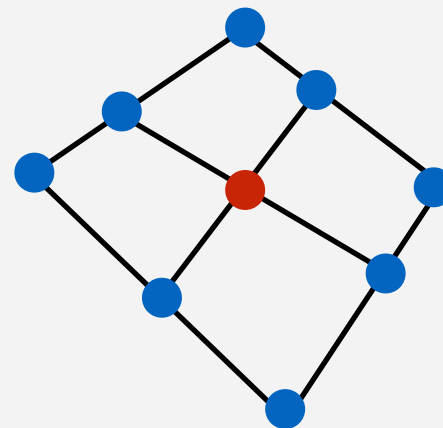
Subdivision concepts

- **Valence/degree** of a vertex
 - the number of edges connected to it
- **Extraordinary** vertex
 - vertex which has a valence different from ordinary vertices
 - by applying subdivision the number of extraordinary points does not change



● valence = 6
interior vertex

● valence $\neq$ 6
boundary vertex

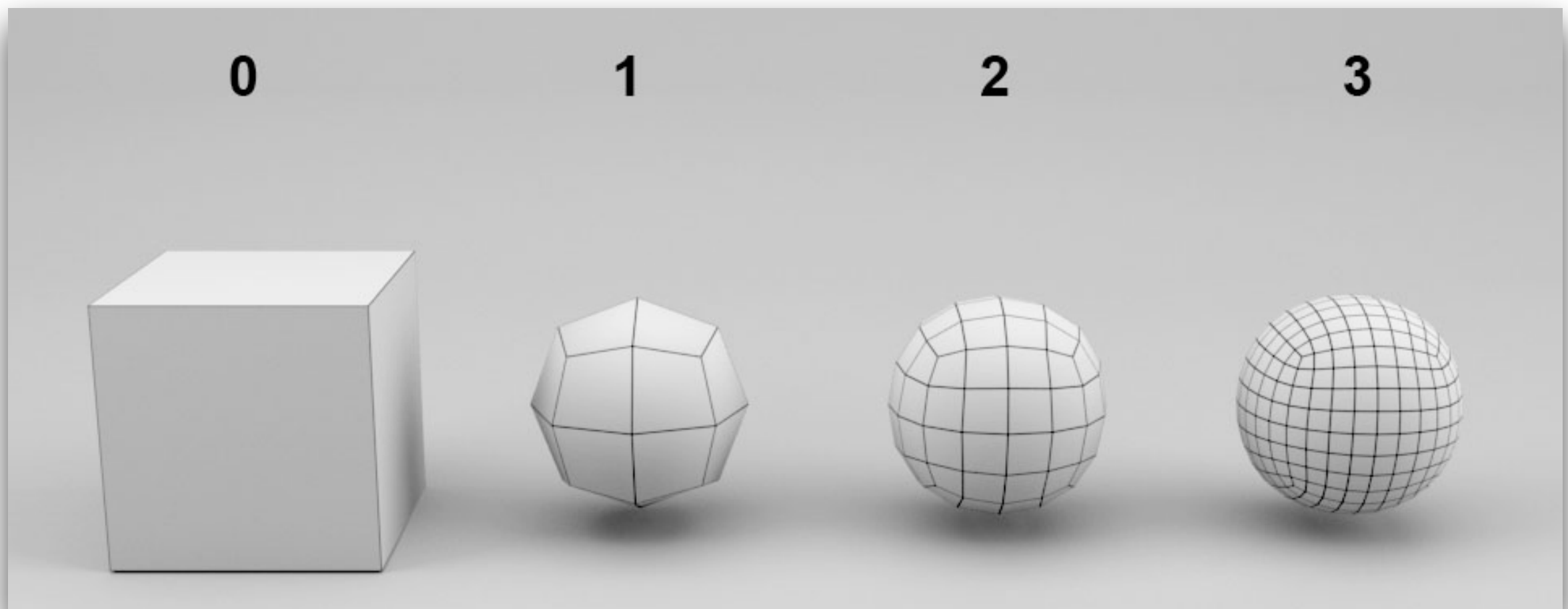


● valence = 4
interior vertex

● valence $\neq$ 4
boundary vertex

Catmull-Clark subdivision

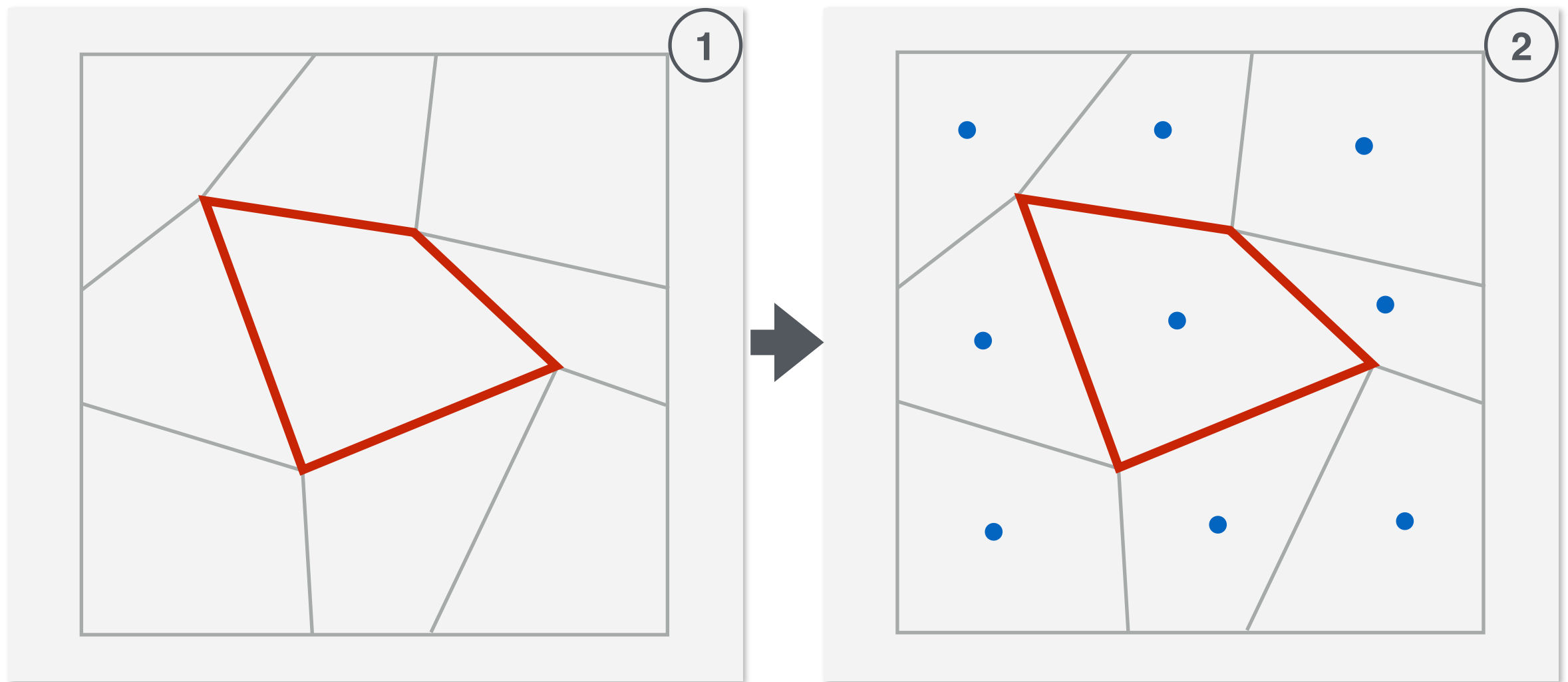
- Works for arbitrary polygonal mesh
- The new mesh (after the first iteration) will consist only of **quadrilaterals**
- **Smoothness** of limit surface:
 - C^1 - at extraordinary vertices
 - C^2 - at ordinary vertices



Catmull-Clark subdivision

Add new face points

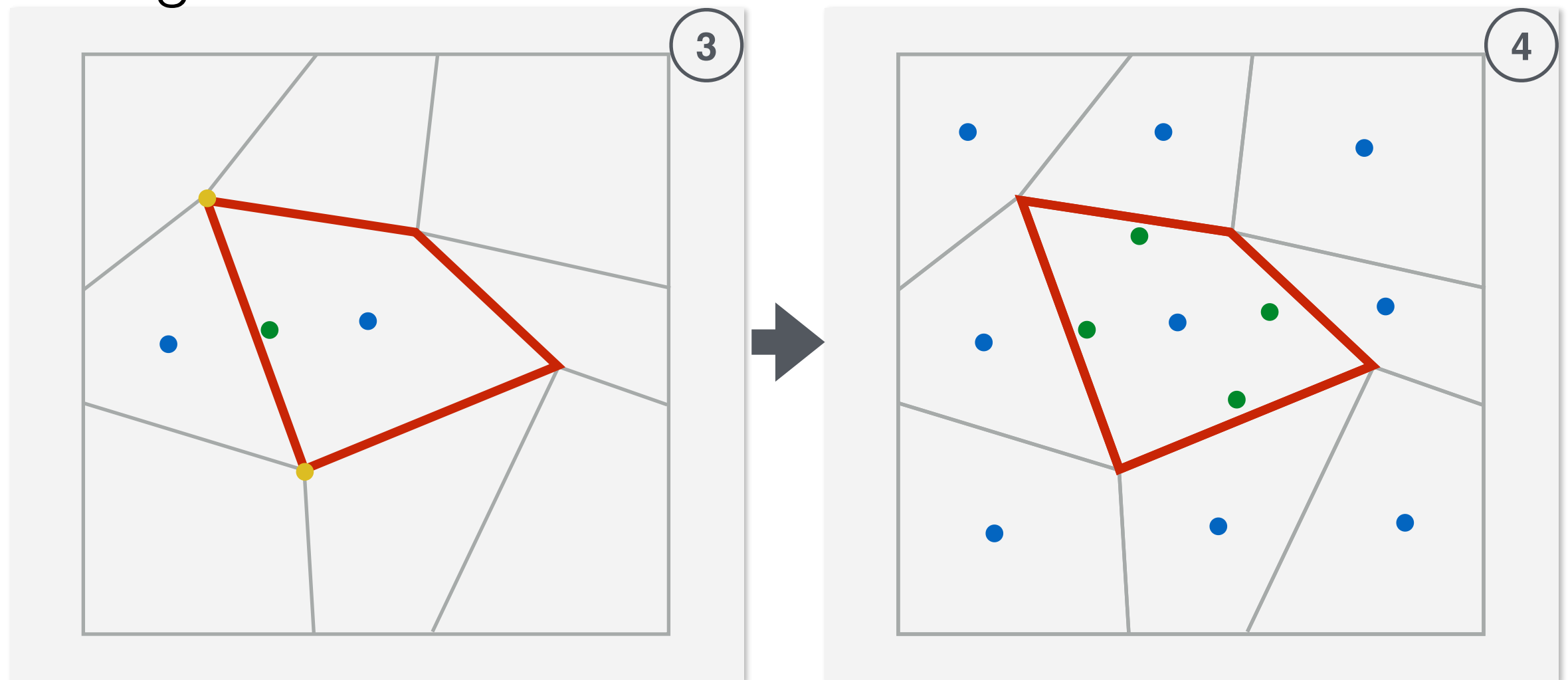
- the average of the points defining the face



Catmull-Clark subdivision

Add new edge points

- the average of the new face points of the faces sharing the edge and the original endpoints of the edge

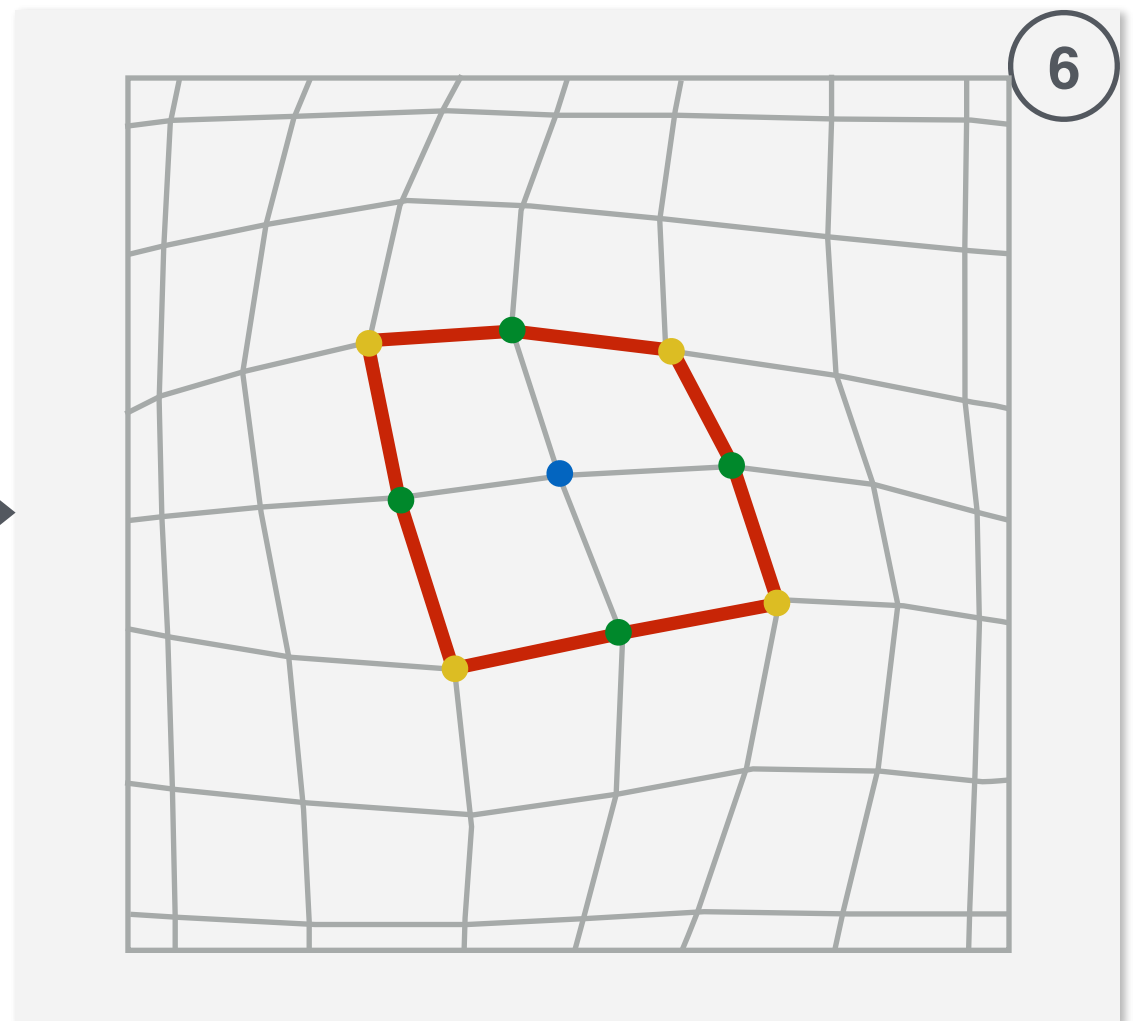
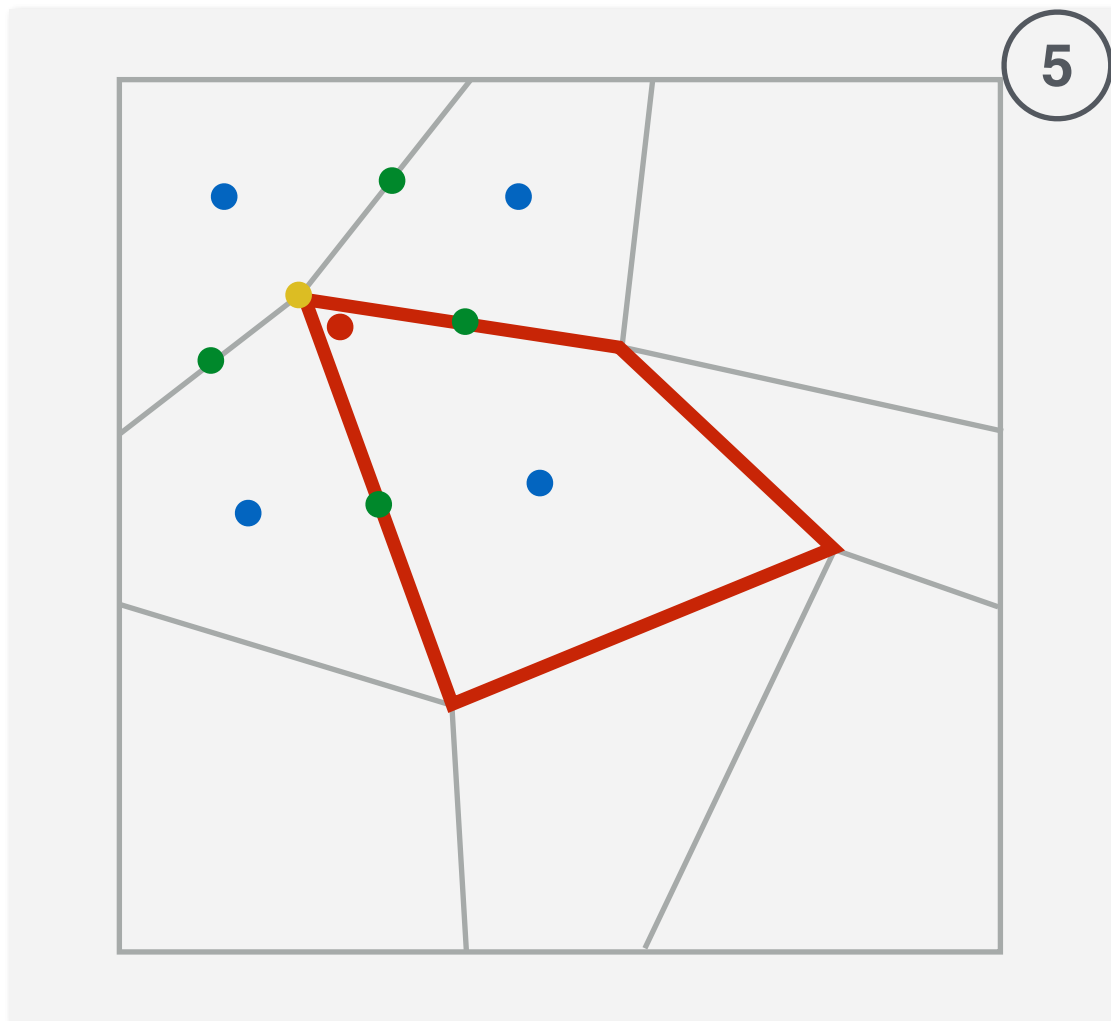


Catmull-Clark subdivision

Move the original point **P**

$$\frac{F + 2R + (n - 3)P}{n}$$

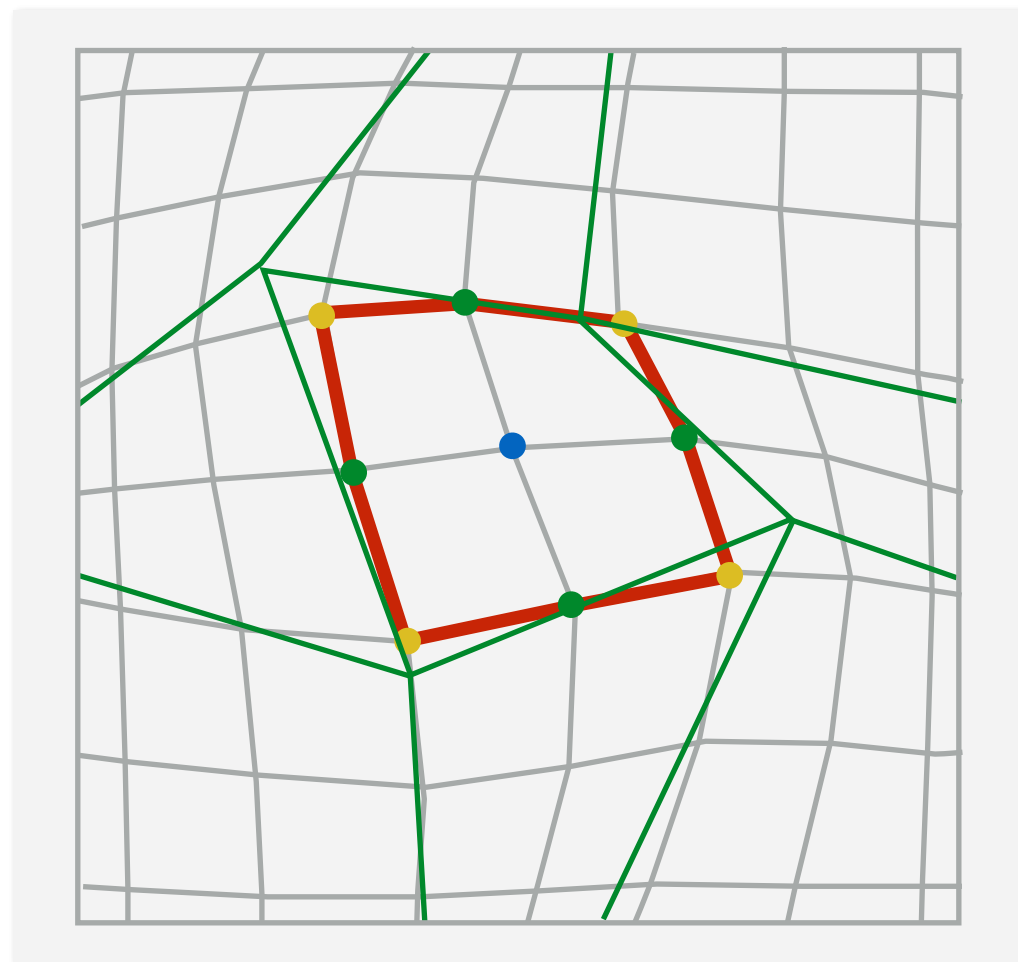
- **F** - the average of all **n** new face points (faces containing point P)
- **R** - the average of all **n** edge midpoints (edges containing point P)
- **P** - the original point



Catmull-Clark subdivision

Modify the topology of the original model

- For each new face point, add new edges connecting the new face point to each new edge point of the face
- Connect each new vertex point to the new edge points
- Define new faces

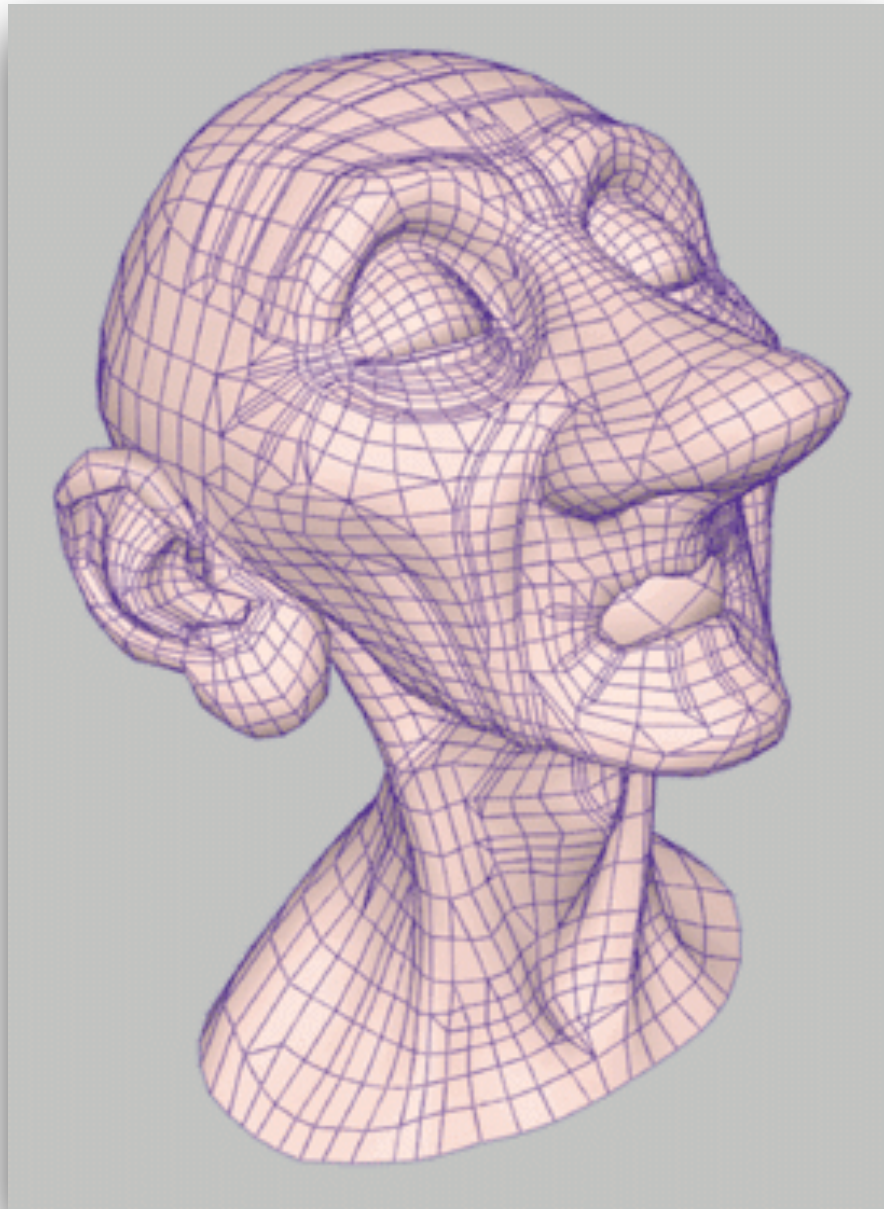


Catmull-Clark subdivision

- **Add new face points** - the average of the points defining the face
- **Add new edge points** - the average of the new face points of the faces sharing the edge and the original endpoints of the edge
- **Move the original point P**
$$\frac{F + 2R + (n - 3)P}{n}$$
 - **F** - the average of all **n** new face points (faces containing point P)
 - **R** - the average of all **n** edge midpoints (edges containing point P)
 - **P** - the original point
- **Modify the topology** of the original model
 - For each new face point, add new edges connecting the new face point to each new edge point of the face
 - Connect each new point to the new edge points
 - Define new faces

Catmull-Clark example

- Pixar - Geri's Game

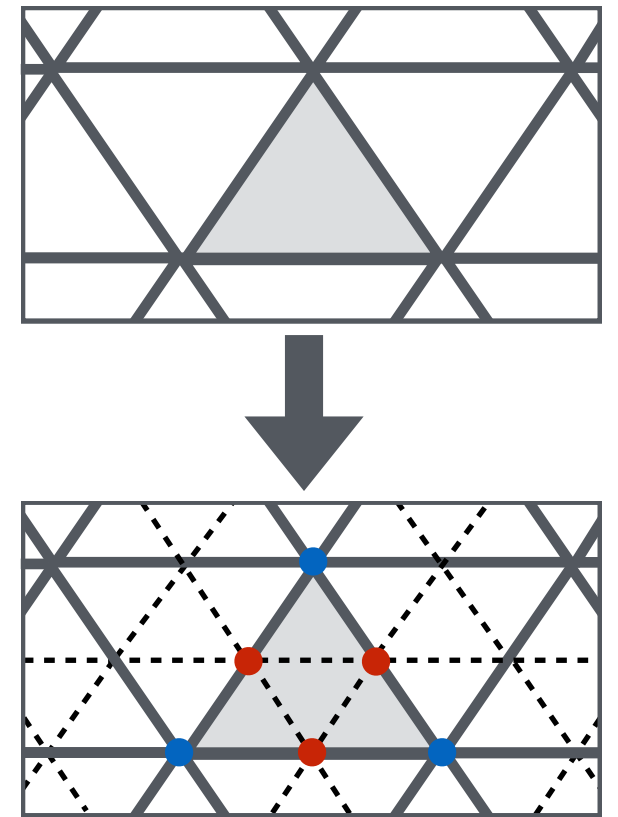


3D object representations

Raw data	Surfaces	Solids	High-level structures
Point cloud	Mesh	Voxel	Scene graph
	Subdivision	CSG	Skeleton
	Parametric	Sweep	
	Implicit		

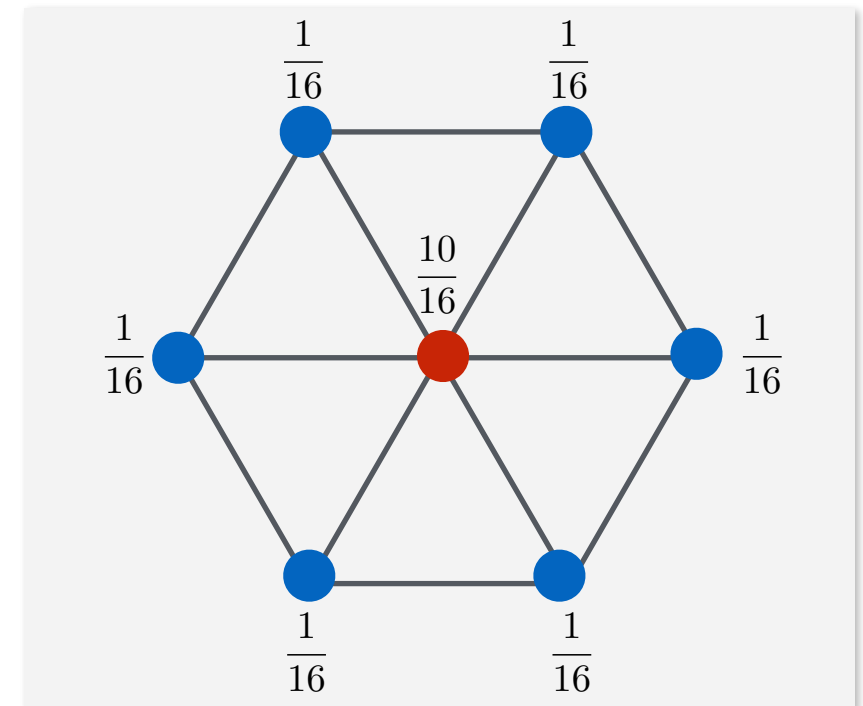
Loop subdivision

- Works on triangles
- Divides each triangle into four smaller ones
- **Smoothness** of limit surface:
 - C^1 - at extraordinary vertices
 - C^2 - at ordinary vertices
- Loop subdivision scheme:
 - **Refinement** - subdivide each triangle into 4 new triangles by splitting each edge and connecting new vertices
 - **Smoothing** - adjust the locations of the existing vertices and of the new inserted vertices



Loop subdivision

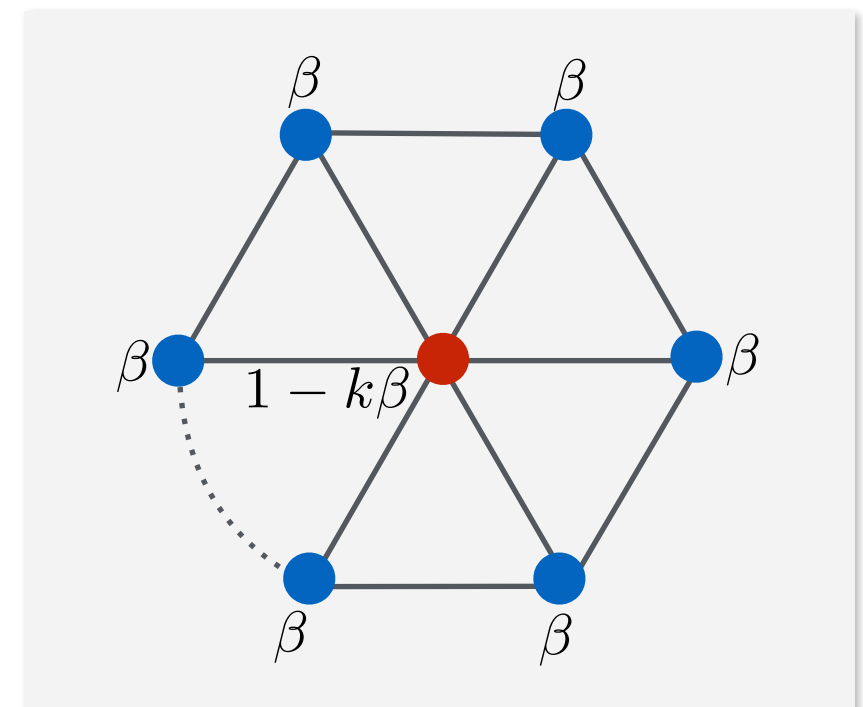
- Move each of the original vertices to new location - weighted average of the original vertex and its neighbours vertices



$$V_{new} = (1 - k\beta) * V_{old} + \sum (\beta * OriginalAdjacentVertex)$$

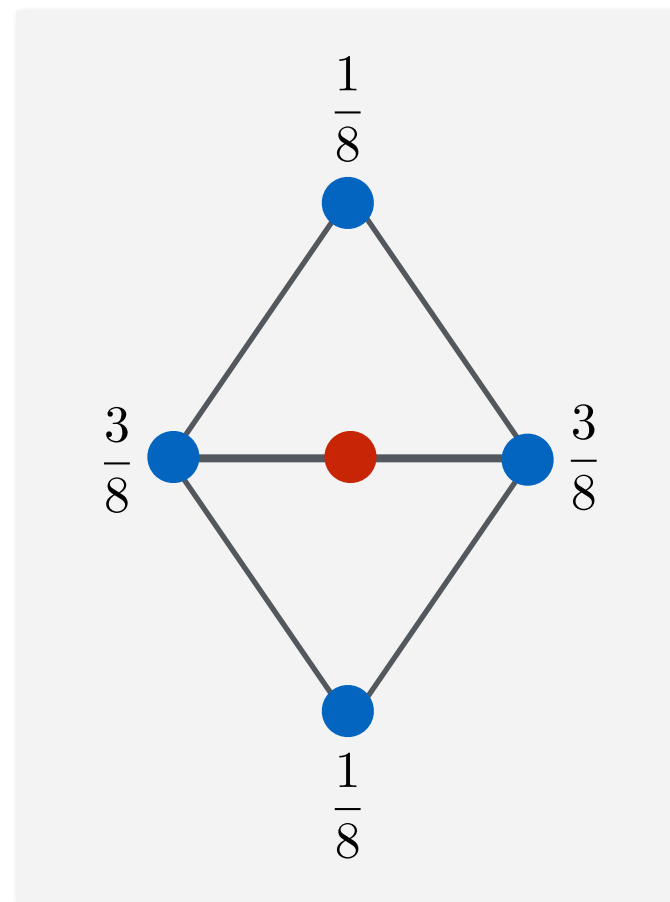
- Original loop: $\beta = \frac{1}{k} \left(\frac{5}{8} - \left(\frac{3}{8} + \frac{1}{4} \cos \frac{2\pi}{k} \right)^2 \right)$

- Warren: $\beta = \frac{3}{8k}, n > 3$
 $\beta = \frac{3}{16}, n = 3$



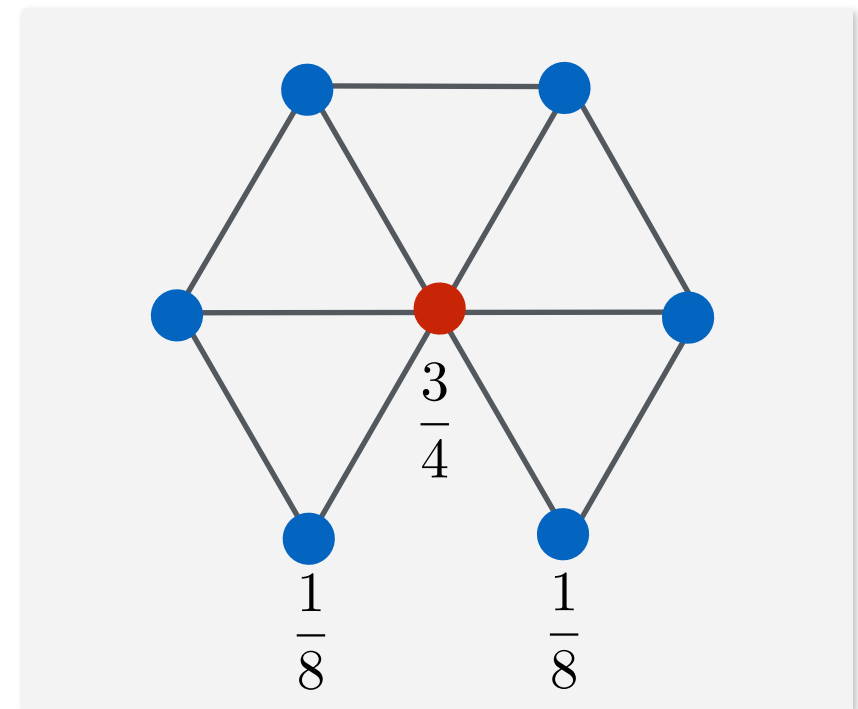
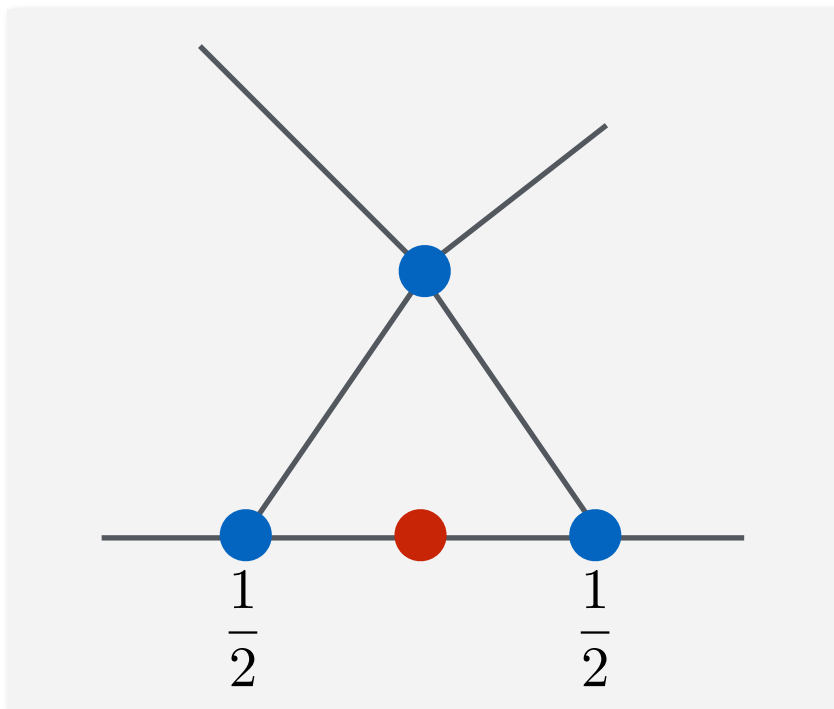
Loop subdivision

- Choose the location for the new vertices as a weighted average of the original vertices in local neighbourhood



Loop subdivision

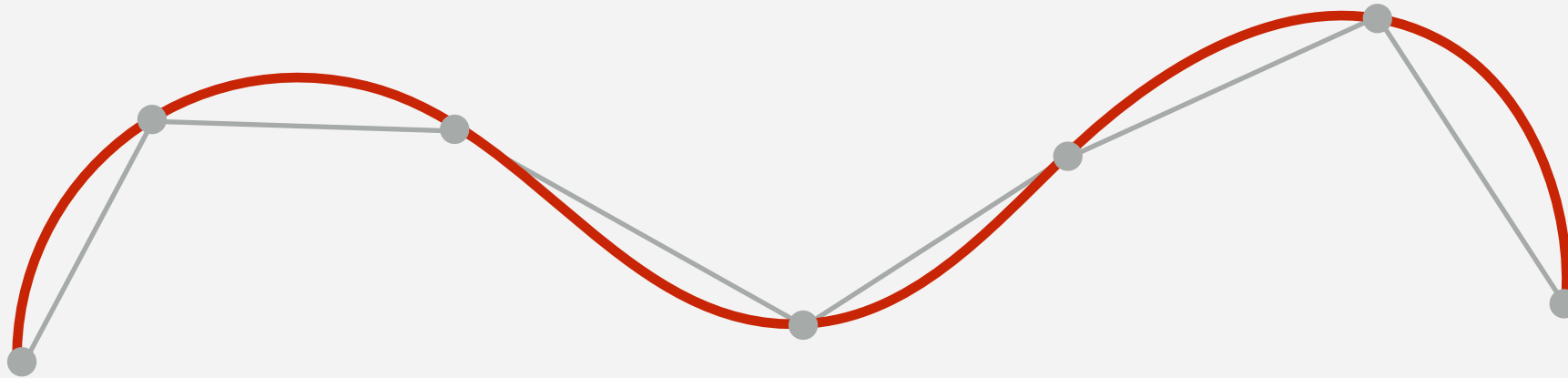
- Rules for crease and boundary:



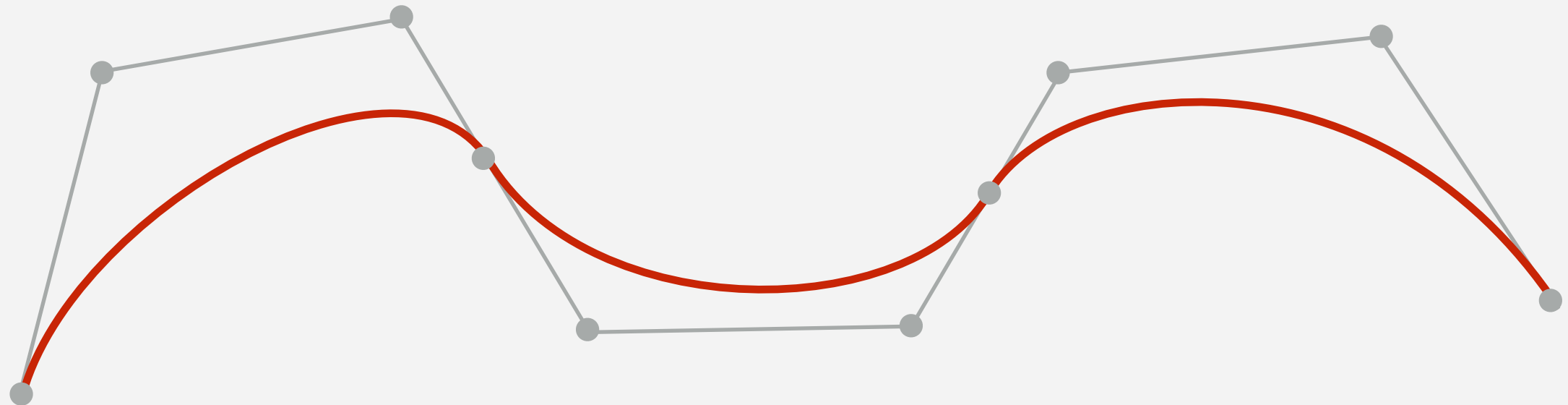
3D object representations

Raw data	Surfaces	Solids	High-level structures
Point cloud	Mesh	Voxel	Scene graph
	Subdivision	CSG	Skeleton
	Parametric	Sweep	
	Implicit		

Splines



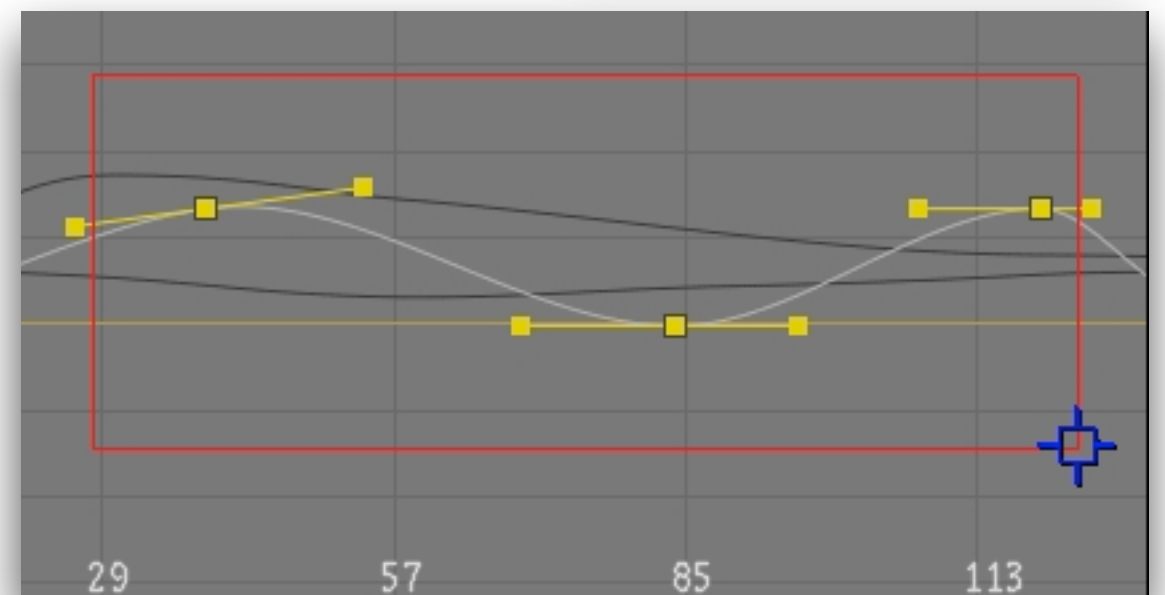
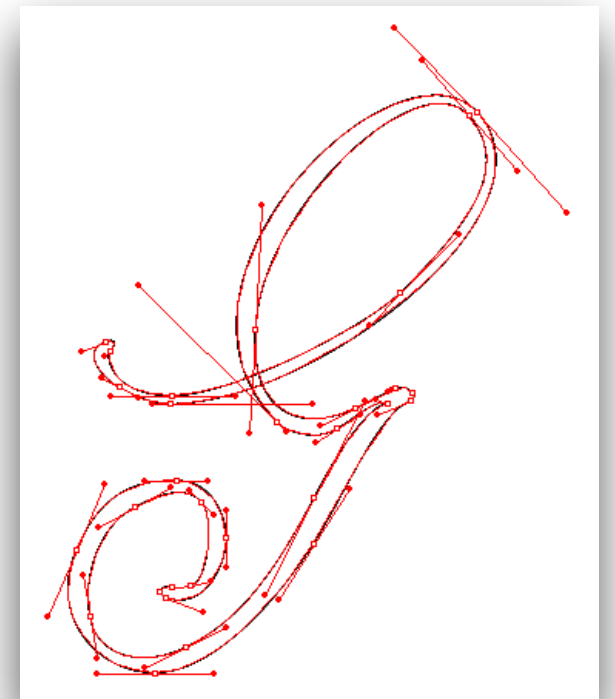
Interpolating - pass through the control points



Approximating - guided by the control points

Splines

- **Spline** - piecewise parametric polynomial function
- Join low degree (mostly cubic) splines
- **Advantage:** local control of curve
- Usage:
 - 2D illustration
 - Fonts
 - 3D modeling
 - Animation (keyframes)



Spline specifications

1. Specifying the set of **boundary conditions** that are imposed on the spline
2. Specifying the **matrix** that characterises the spline
3. Specifying the set of **blending functions** that determine how geometric constraints on the curve are combined to calculate positions along the curve path.

Spline specification: Boundary conditions

- Considering a parametric cubic polynomial spline representation:

$$Q(t) = at^3 + bt^2 + ct + d, 0 \leq t \leq 1$$

- Set the boundary conditions for:
 - endpoint coordinates: $Q(0)$ and $Q(1)$
 - first derivatives at the endpoints: $Q'(0)$ and $Q'(1)$
- These four boundary conditions are sufficient to determine the values of the four coefficients a , b , c , and d

Spline specification: Matrix form

- **M_{geom}** - four-element column matrix containing the geometric constraint values (boundary conditions)
- **M_{spline}** - 4x4 matrix that transforms the geometric constraint values to the polynomial coefficients and provides a characterisation for the spline curve; useful for transforming from one spline representation to another

$$Q(t) = at^3 + bt^2 + ct + d, 0 \leq t \leq 1$$

$$Q(t) = \begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}$$

$$Q(t) = T \cdot M_{spline} \cdot M_{geom}$$

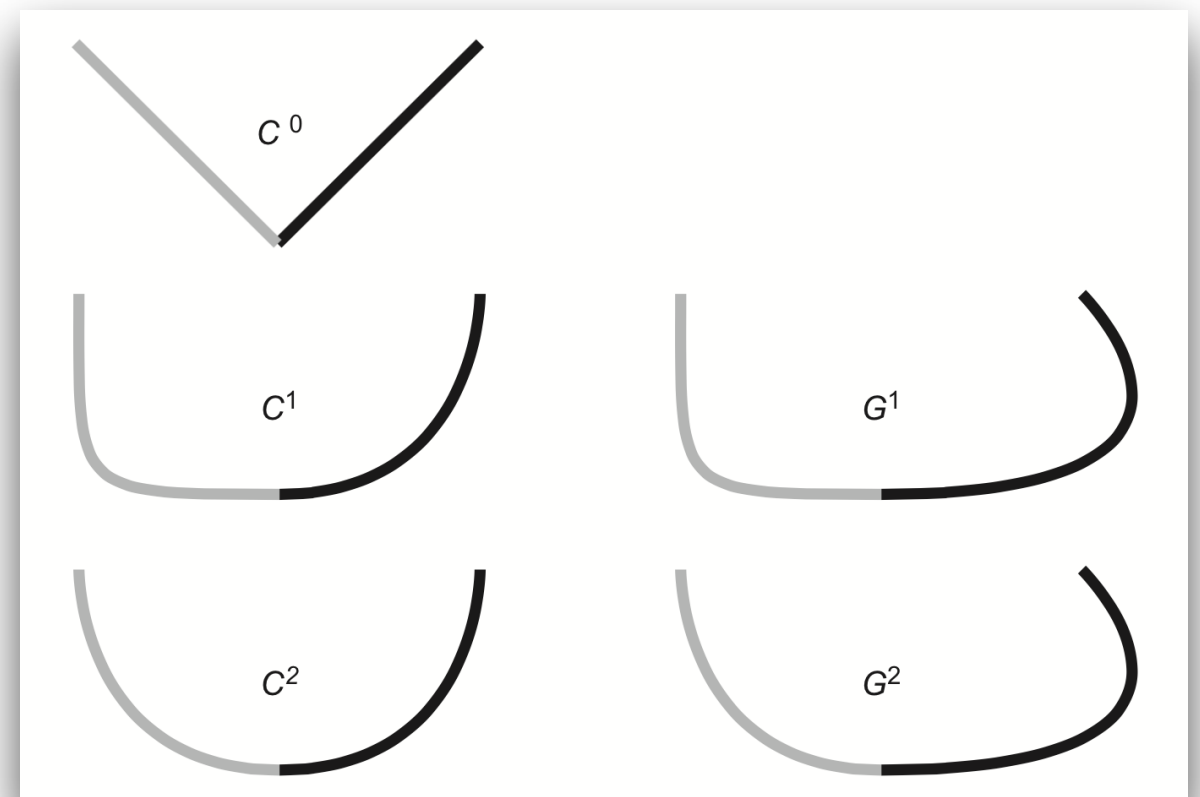
Spline specification: blending functions

- The curve is a weighted sum of the elements of the geometry matrix
- The weights are cubic polynomials of t , called **blending functions**

$$B = T \cdot M_{spline}$$

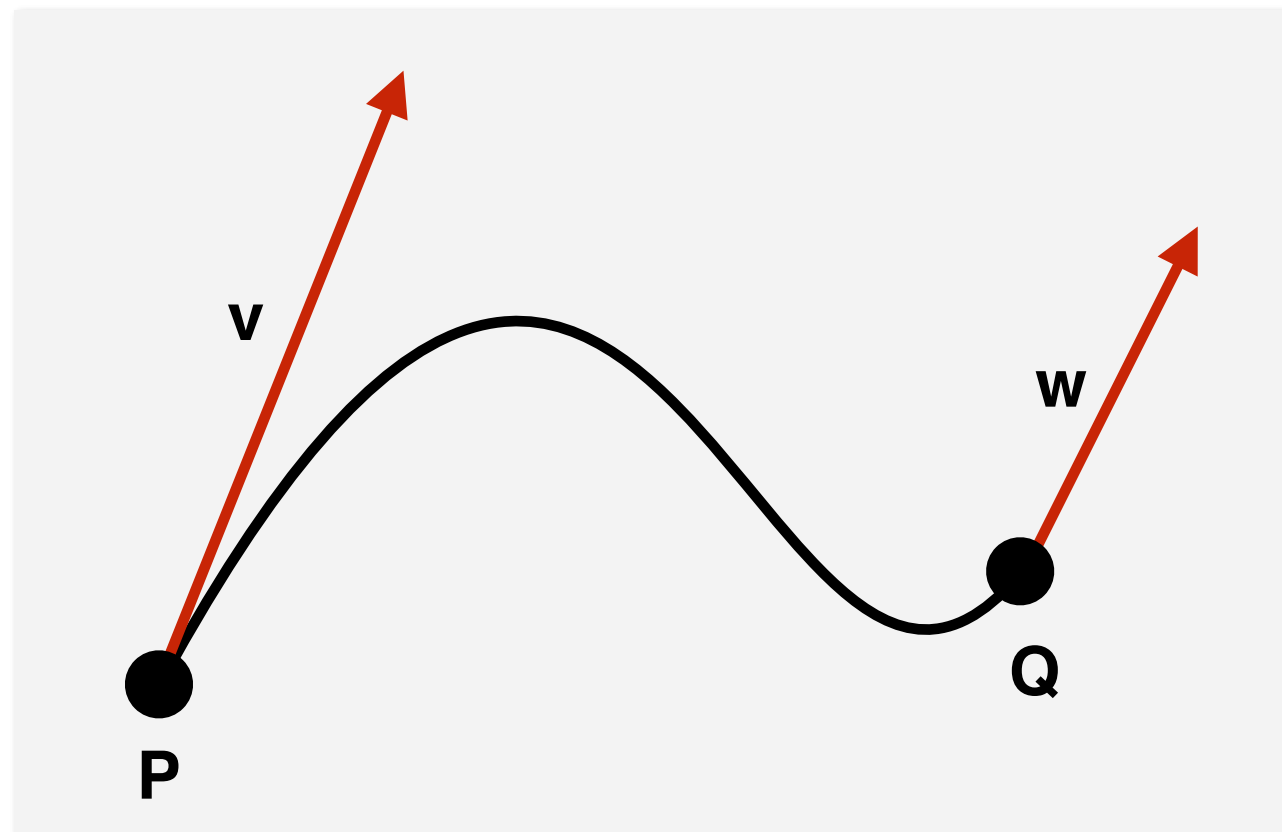
Continuity at joints

- **C^0 continuity** - spline segments are connected at joints
- **C^1 continuity** - spline segments share same first derivative at joints
- **C^n continuity** - spline segments share same n^{th} derivative at joints
- **G^0** - Same as C^0
- **G^1** - Weaker than C^1 , the tangents lengths can be different



Hermite spline

- Named after the French mathematician Charles Hermite
- Interpolating piecewise cubic polynomial spline
- Specified by the **endpoints** and the **tangent vectors** at endpoints (first derivative)



Hermite curve

$$Q(t) = \begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix} \cdot M_{spline} \cdot M_{geom}$$



$$Q(0) = P = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix} \cdot M_H \cdot G_H$$

$$Q(1) = Q = \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} \cdot M_H \cdot G_H$$

$$Q'(0) = \mathbf{v} = \begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix} \cdot M_H \cdot G_H$$

$$Q'(1) = \mathbf{w} = \begin{bmatrix} 3 & 2 & 1 & 0 \end{bmatrix} \cdot M_H \cdot G_H$$



$$\begin{bmatrix} P \\ Q \\ v \\ w \end{bmatrix} = G_H = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{bmatrix} \cdot M_H \cdot G_H$$

Hermite curve

- Hermite geometry vector/matrix

$$G_H = \begin{bmatrix} P \\ Q \\ v \\ w \end{bmatrix}$$

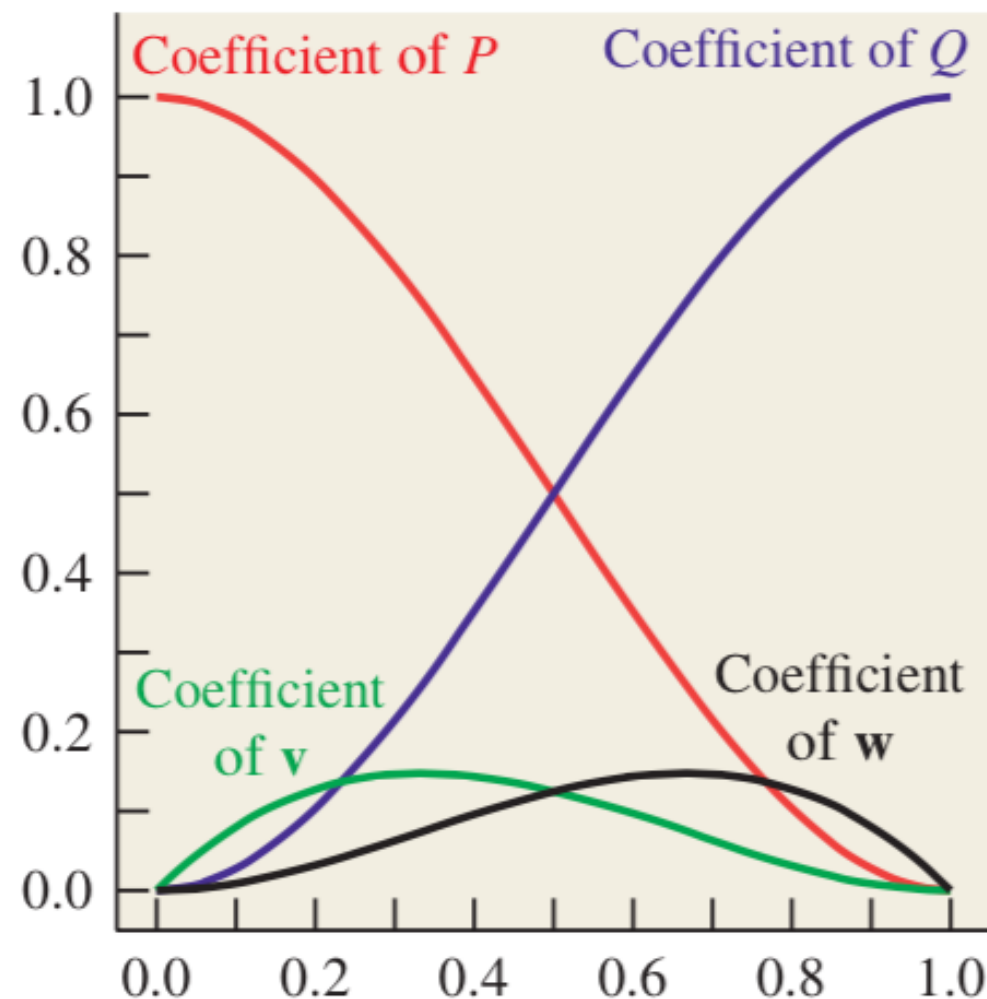
- Hermite basis matrix

$$M_H = \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

Hermite spline

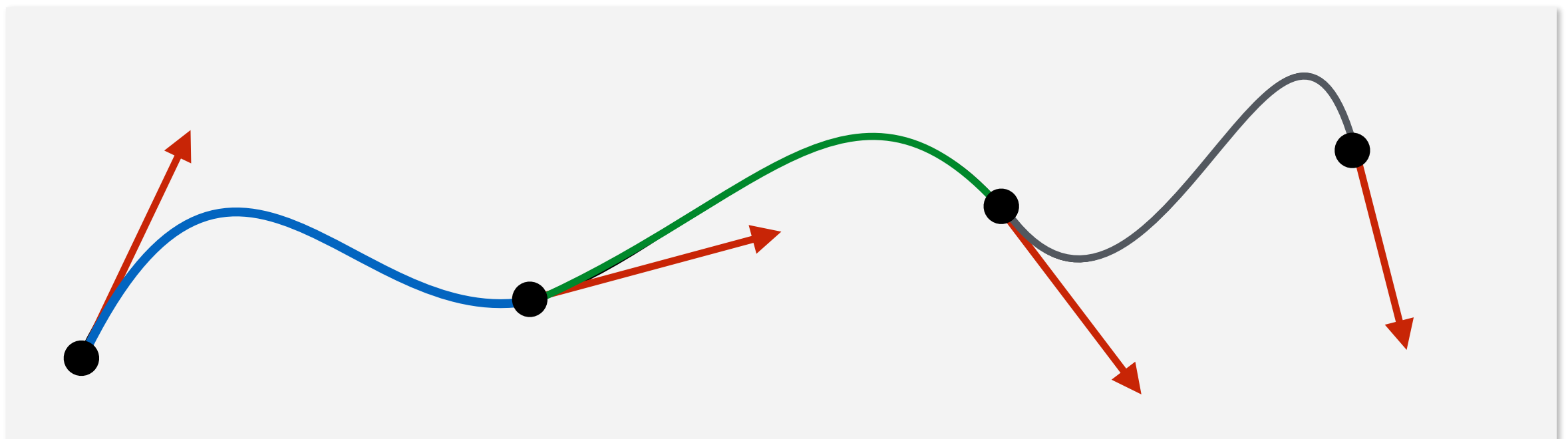
- The four polynomials are called **Hermite basis (blending) functions** and tell how the inputs are combined to make the curve

$$f(t) = (2t^3 - 3t^2 + 1)P + (-2t^3 + 3t^2)Q + (t^3 - 2t^2 + t)\mathbf{v} + (t^3 - t^2)\mathbf{w}$$



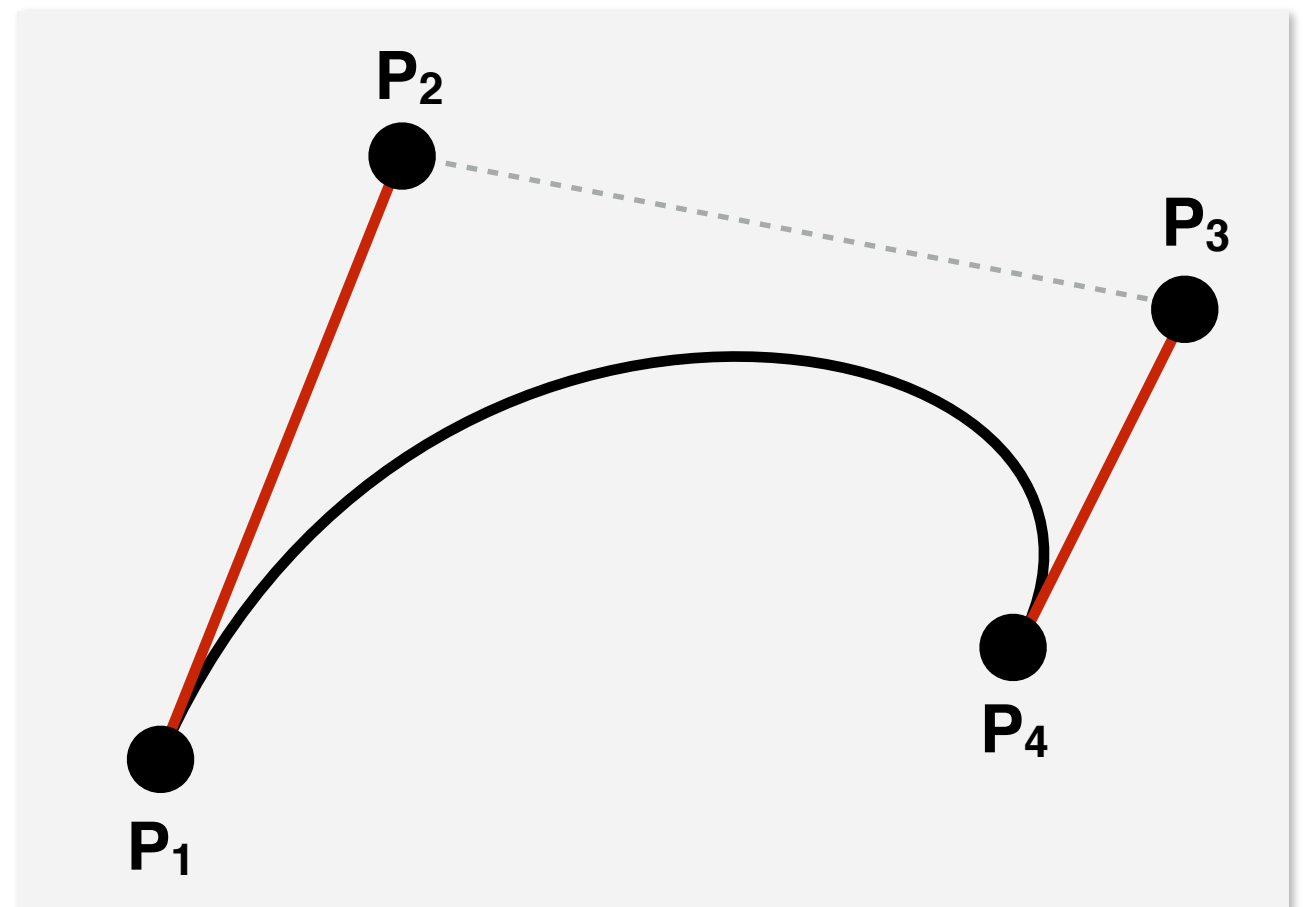
Hermite spline

- Link a chain of curve segments into a C^1 spline:
 - use for the end of the segment and the beginning of the next one the same values for the position and derivative
- For a set of n control points a cubic Hermite spline contains $(n-2)/2$ cubic segments
- Advantages:
 - **local control** over the shape
 - C^1 **continuity**



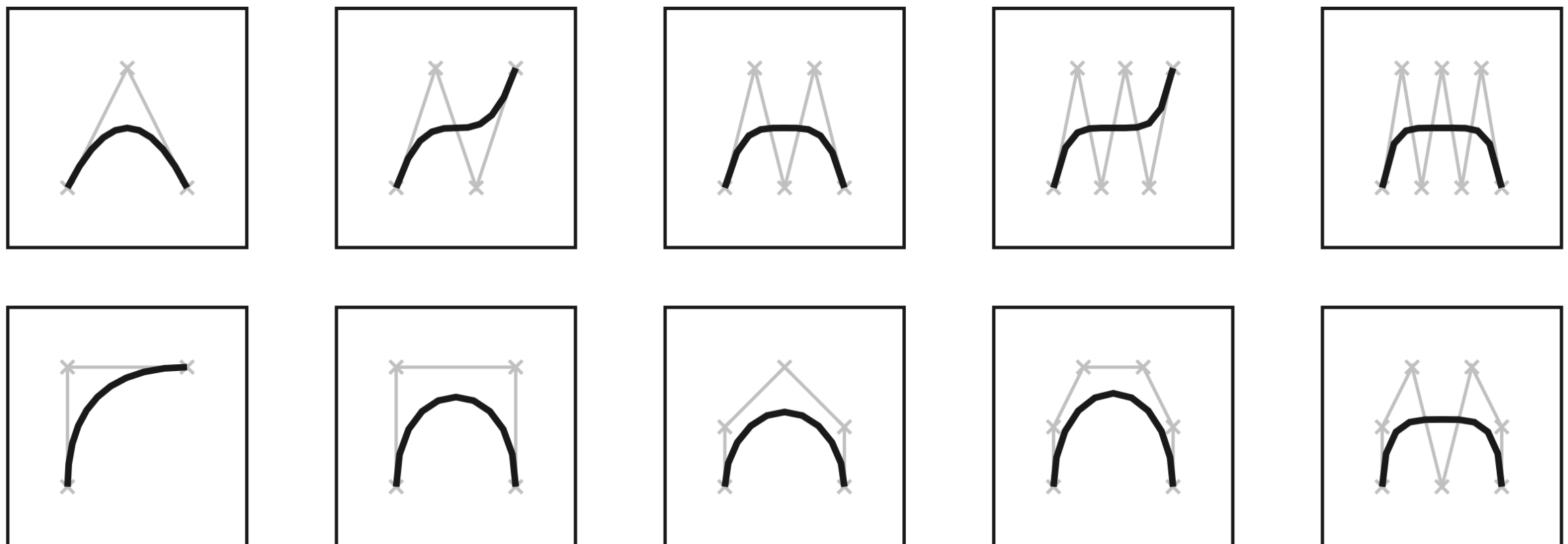
Bézier curve

- Firstly developed by **Paul de Casteljaou** at Citroën
- Named after **Pierre Bézier** (from Renault)
- Widely used in:
 - CAD
 - font definition
 - animation
 - graphic design



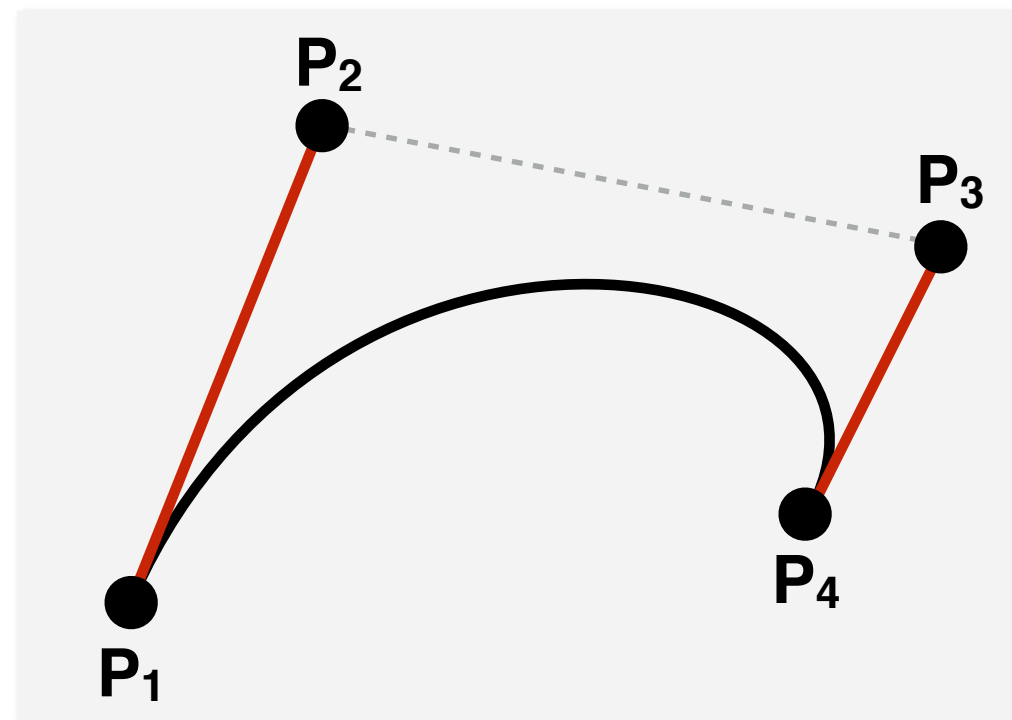
Bézier curve

- A curve of degree d is controlled by $d + 1$ control points and approximates its control points
- The curve interpolates its first and last control points and the shape is influenced by the other control points.
- Properties:
 - The curve is bounded by the convex hull of the control points
 - The curves are symmetric, by reversing the order of the control points we get the same curve



Bézier curve

- **Cubic** Bézier curves - can define a curve by specifying 2 endpoints and 2 additional control points
- The curve starts at P_1 , finishes at P_4 , and first derivative at the beginning of the curve is $3(P_2 - P_1)$ and at the end of the curve is $3(P_4 - P_3)$



Bézier curve

$$f(t) = (-t^3 + 3t^2 - 3t + 1)P_1 + (3t^3 - 6t^2 + 3t)P_2 + (-3t^3 + 3t^2)P_3 + t^3P_4$$

$$M_B = \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

- The blending functions for Bézier curves have a special form that works for all degrees - known as *Bernstein basis polynomials*

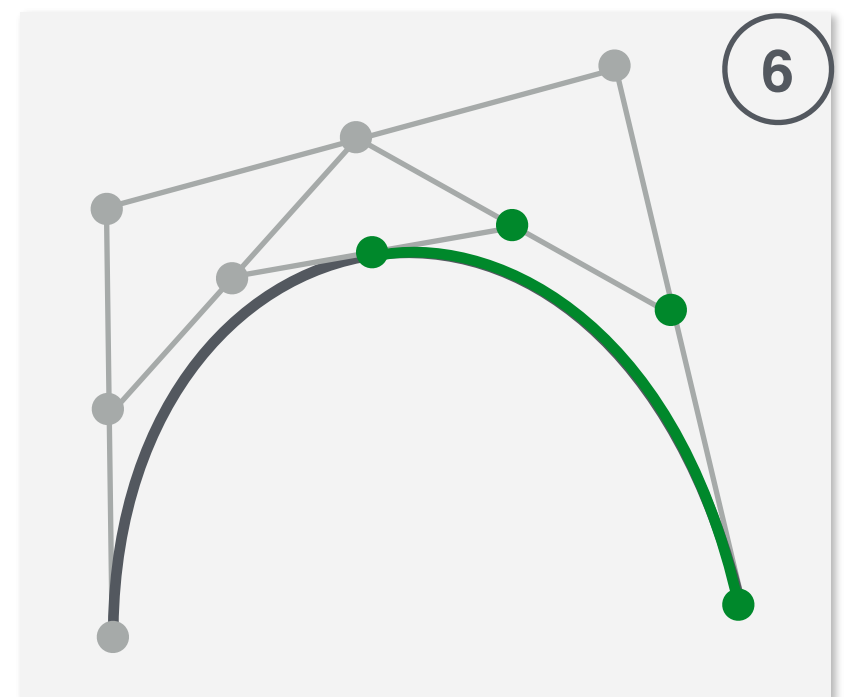
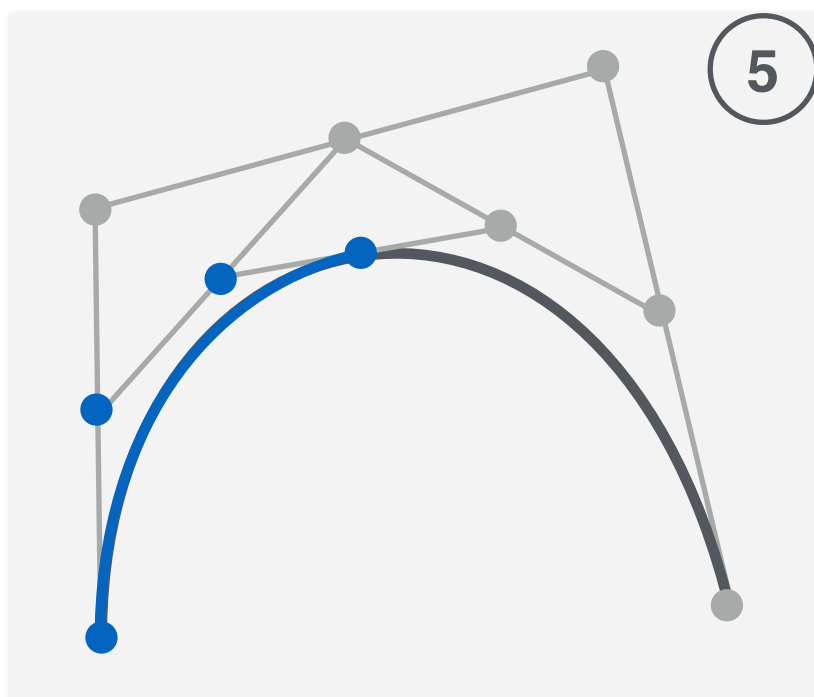
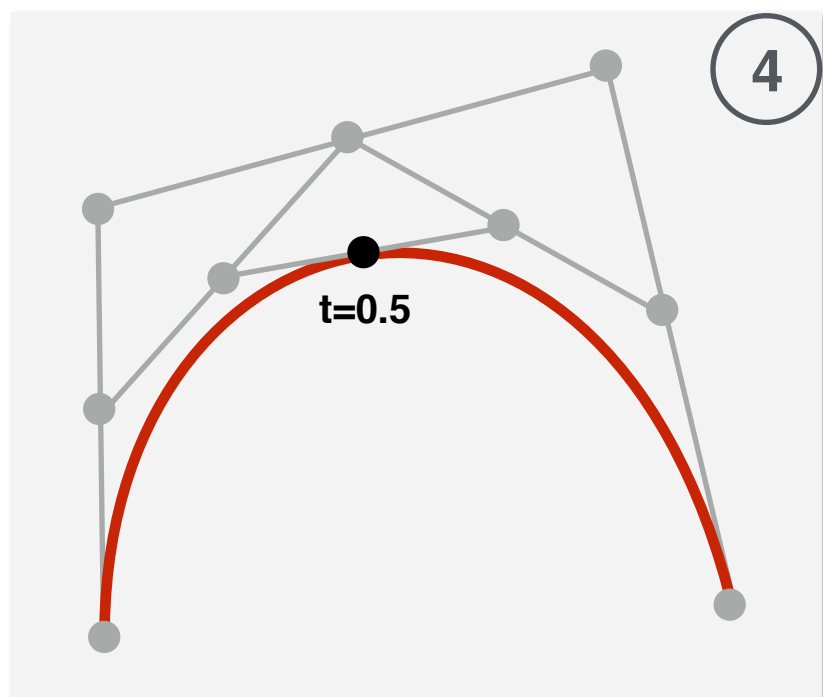
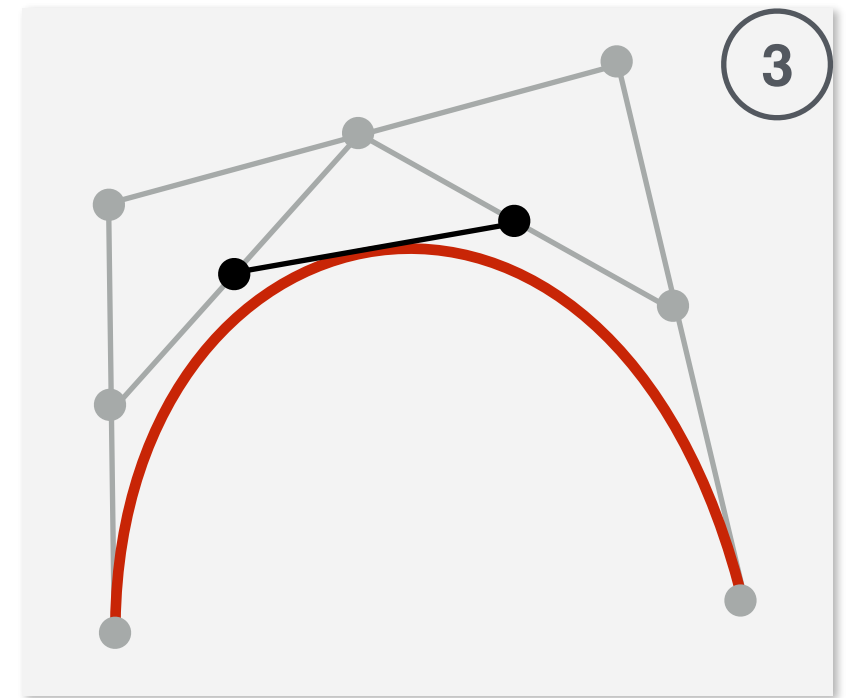
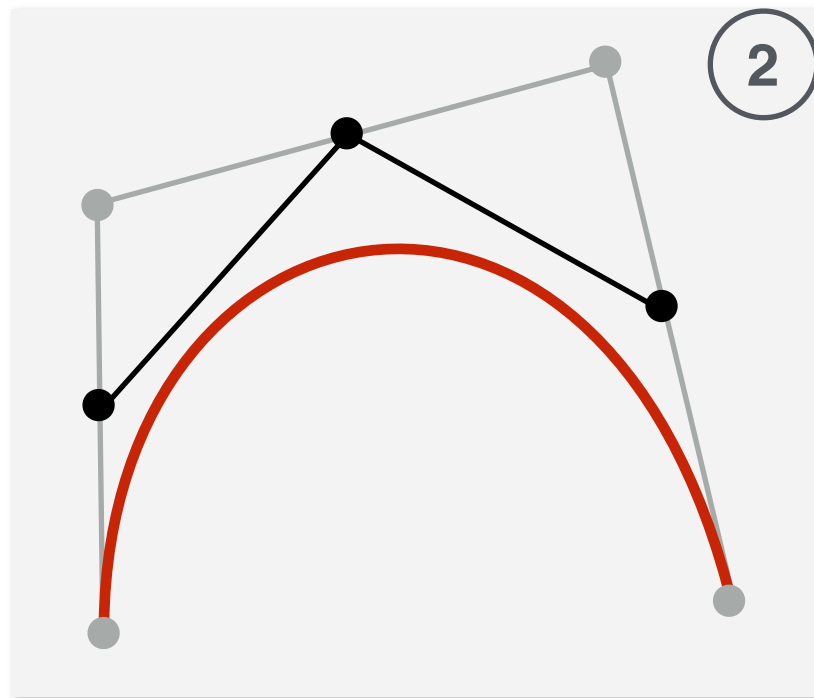
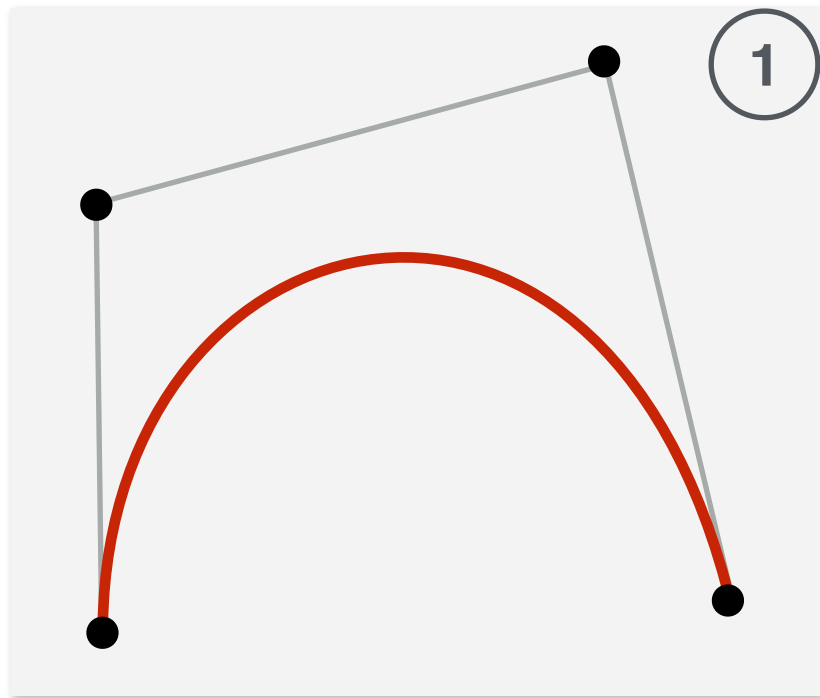
$$b_{k,n}(t) = C(n, k)t^k(1 - t)^{(n-k)}$$

$$C(n, k) = \frac{n!}{k!(n - k)!}$$

De Casteljau Algorithm

- Uses a sequence of linear interpolations to compute the positions along the Bézier curve of arbitrary order.
- The intermediate points computed during the de Casteljau algorithm form the new control points of the new, smaller segments

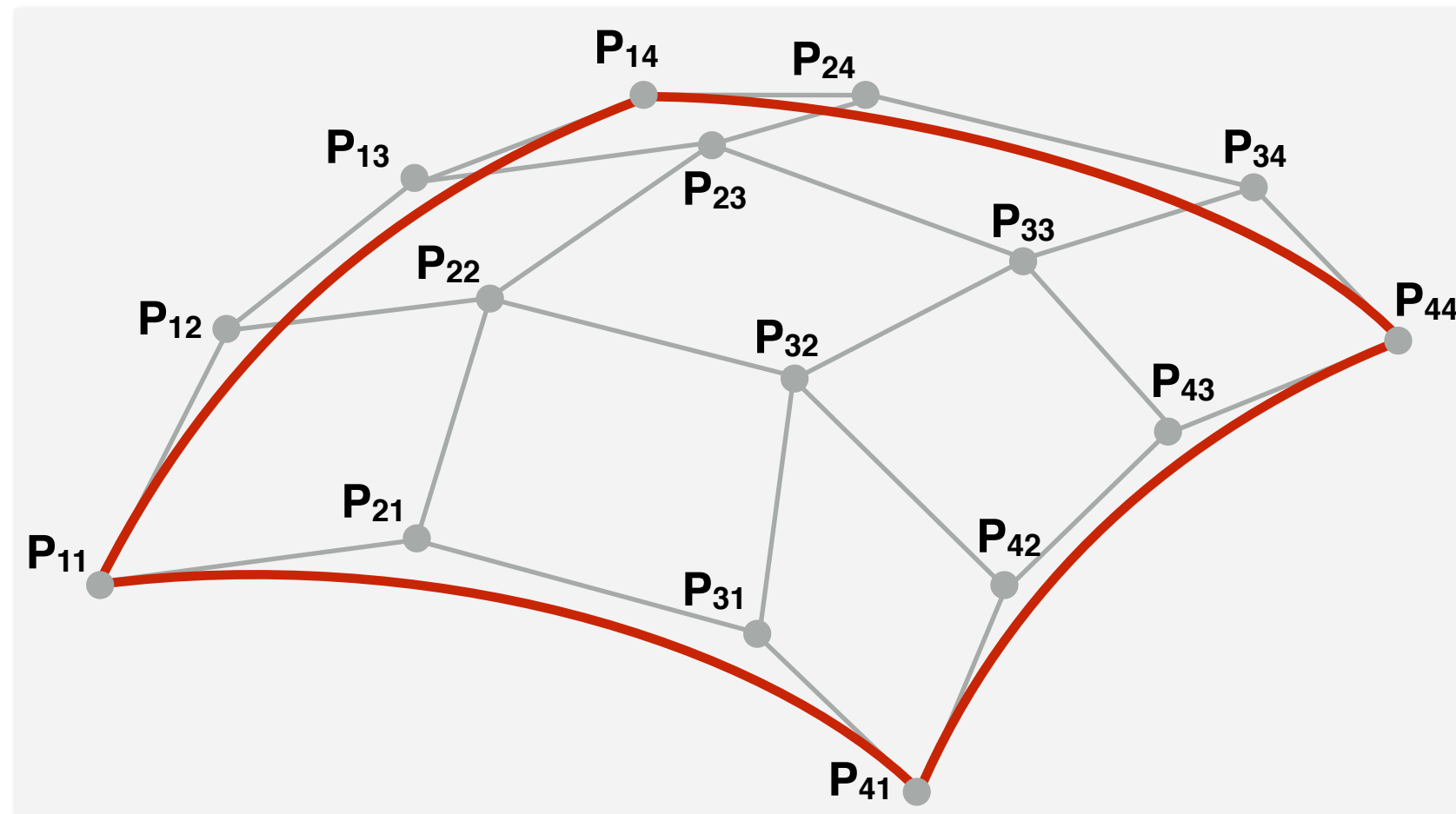
De Casteljau Algorithm



De Casteljau Algorithm

```
point bezierPoint(int n, point[] controlPt, float t)
{
    point deCasPt[n+1];
    for (i=0; i <= n; i++)
        deCasPt[i] = controlPt[i];
    for (r=1; r <= n; r++) {
        for (i=0; i <= n-r; i++) {
            deCasPt[i] = (1-t)*deCasPt[i] + t*deCasPt[i+1];
        }
    }
    return deCasPt[0];
}
```

Bézier patches

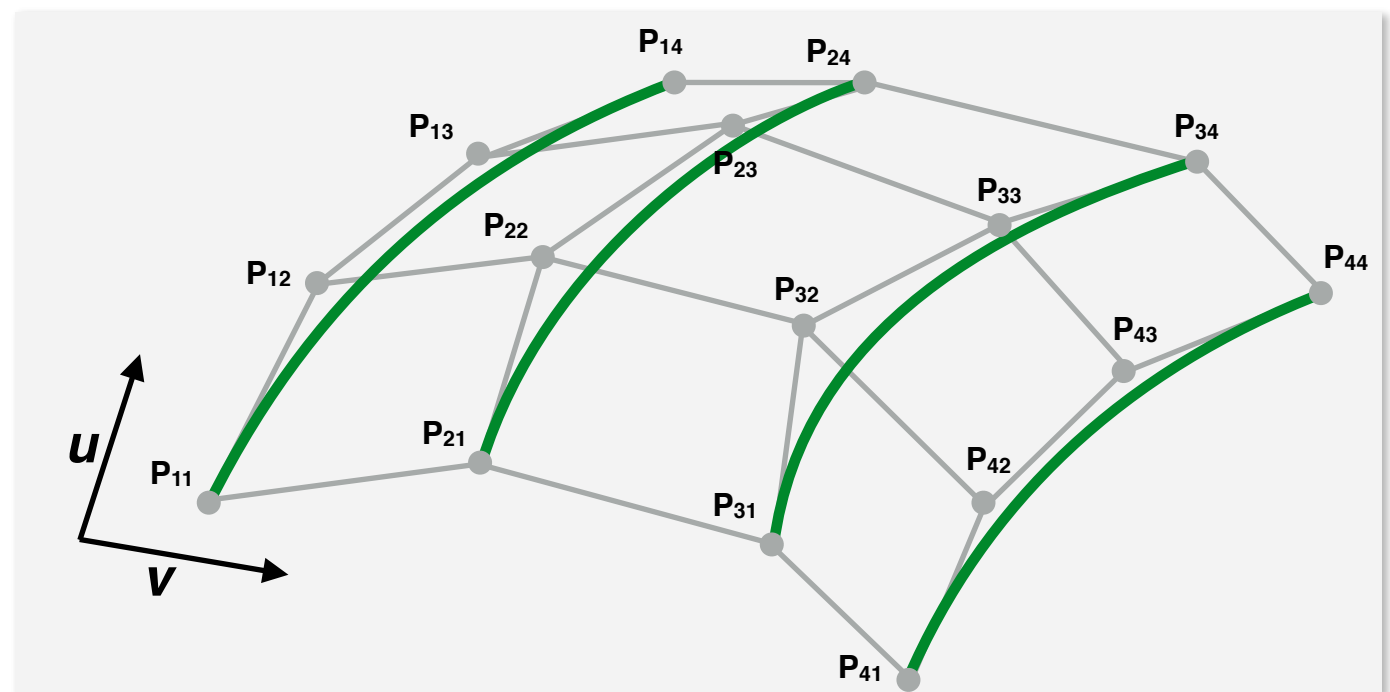
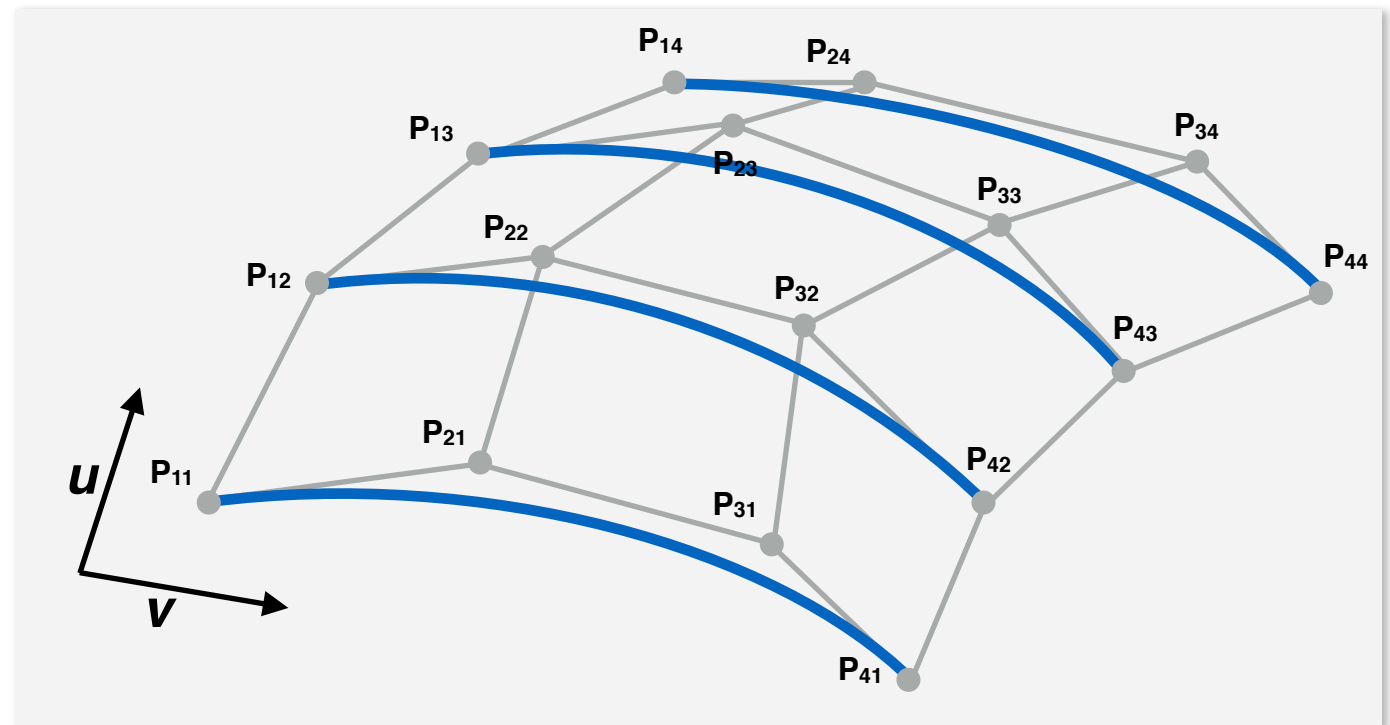


$$Q(u, v) = U \cdot M_B \cdot G_B \cdot M_B^T \cdot V^T$$

$$Q(u, v) = U \cdot M_B \cdot \begin{bmatrix} P_{11} & P_{12} & P_{13} & P_{14} \\ P_{21} & P_{22} & P_{23} & P_{24} \\ P_{31} & P_{32} & P_{33} & P_{34} \\ P_{41} & P_{42} & P_{43} & P_{44} \end{bmatrix} \cdot M_B^T \cdot V^T$$

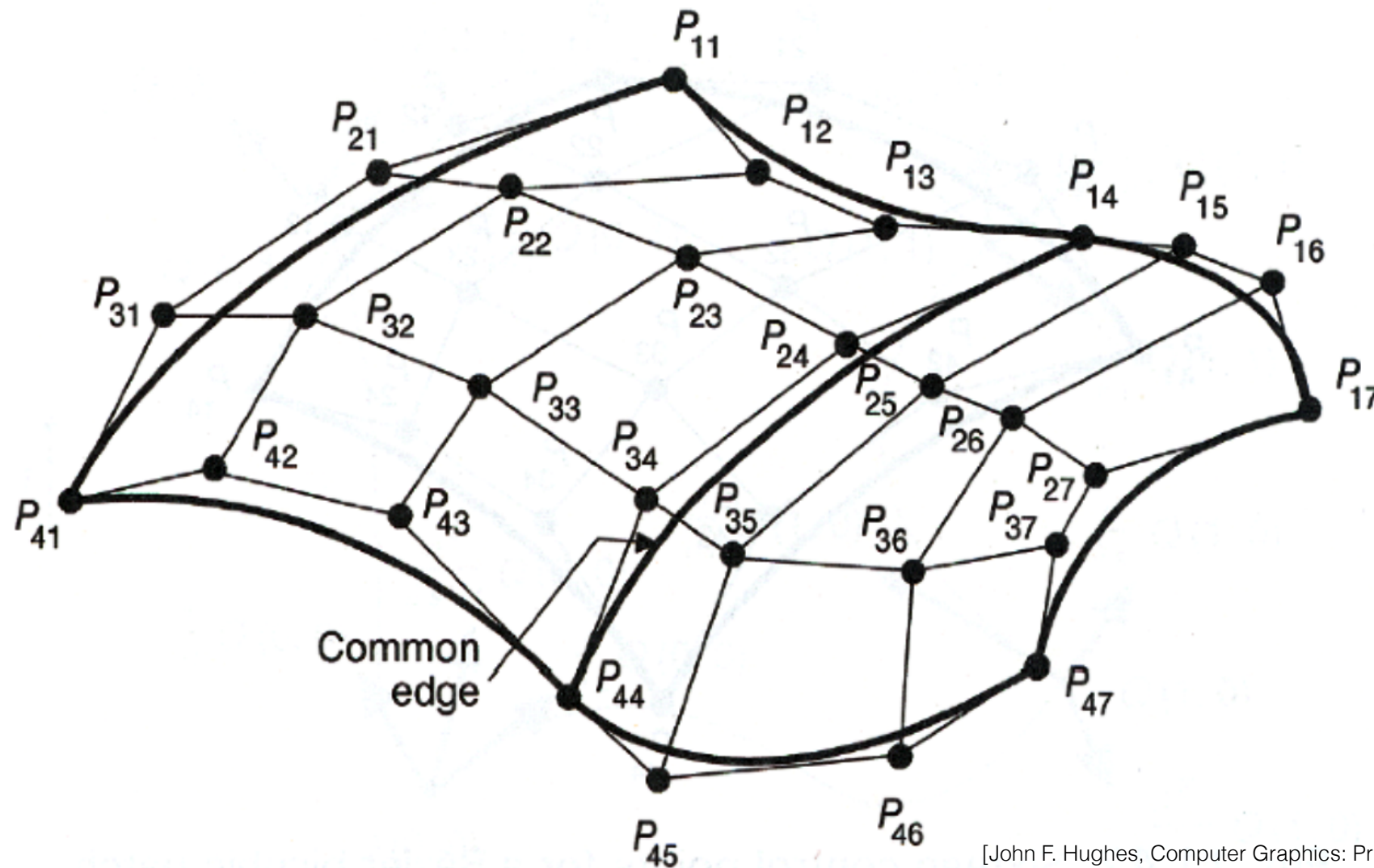
Bézier patches

- Plot each curve of constant u by varying v
- Plot each curve of constant v by varying u
- u and v are varying over the interval from 0 to 1



Join Bézier patches

- C^0 - obtained by matching control points at the boundary
- C^1 - obtained by choosing control points along a straight line across the boundary and by maintaining a constant ratio of collinear line segments for each set of specified control points across section boundaries

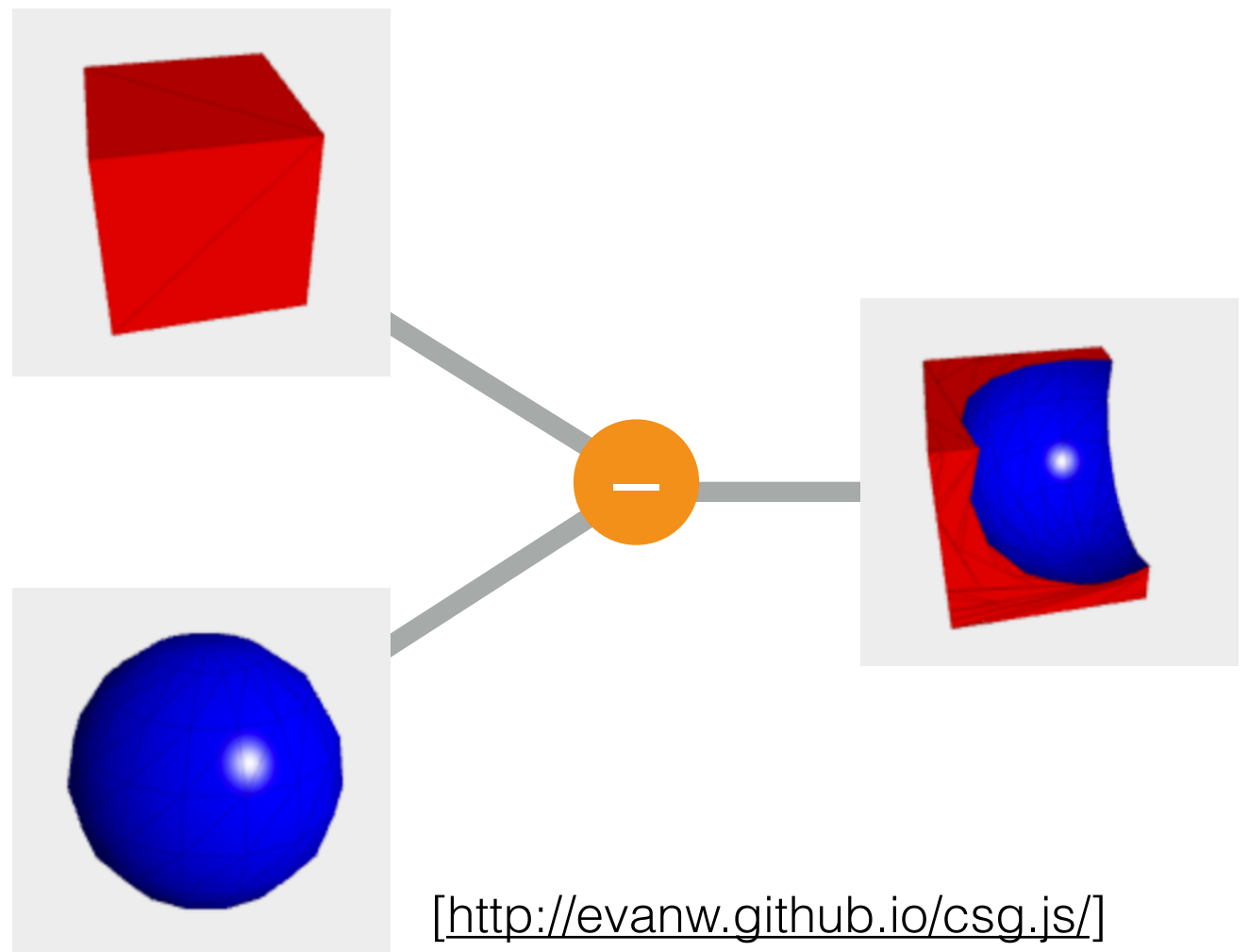


3D object representations

Raw data	Surfaces	Solids	High-level structures
Point cloud	Mesh	Voxel	Scene graph
	Subdivision	CSG	Skeleton
	Parametric	Sweep	
	Implicit		

Constructive Solid Geometry (CSG)

- Create complex 3D objects using boolean operators on simple 3D objects
- Represent CSG objects using binary trees
- Operations
 - Union
 - Difference
 - Intersection



[<http://evanw.github.io/csg.js/>]

Constructive Solid Geometry (CSG)

